

Skalierbarkeit von Mean-Embeddings als Dimensionsreduktion für Reinforcement-Learning Algorithmen in Schwarmsystemen

Robin Ben Grellner

5. Juli 2026



Masterarbeit

Betreuender Professor: Prof. Dr. David Spieler
Zweitkorrektor: Prof. Dr. Christoph Böhm

Inhaltsverzeichnis

I Abkürzungsverzeichnis

1	Einleitung	1
1.1	Motivation	1
1.2	Forschungsfragen und Hypothesen	1
1.3	Ziel und Aufbau der Arbeit	2
2	Hintergrund	3
2.1	Reinforcement Learning: Grundlagen	3
2.2	Deep Reinforcement Learning: TRPO und PPO	6
2.3	Multi Agent Reinforcement Learning	8
2.4	Schwarmsysteme	12
2.5	Permutationsinvarianz und Kernel Mean Embeddings	15
2.6	Repräsentationsdimnesion und untere Schranken	20
2.7	Mean Embeddings nach Hüttenrauch	22
3	Verwandte Arbeiten	23
3.1	Skalierungsinvariante Architekturen in MARL	23
3.2	Dimensionsreduktion und Repräsentationskapazität	26
3.3	Einordnung und Forschungslücke	28
4	Implementierung	29
4.1	Architekturentscheidungen	29
4.2	Mean-Embedding Feature Extractor	30
4.3	Umgebungen und Observation modes	31
4.4	Integration in Stable-Baselines3	32
4.5	Analyse- und Evaluationswerkzeuge	33
5	Design der Experimente	33
5.1	Operationalisierung der Forschungsfragen	33
5.2	Aufgaben und Versuchsbedingungen	33
5.3	Konfigurationsmatrix	34
5.4	Trainingsprotokoll und Hyperparameter	34
5.5	Evaluations- und Metrikprotokoll	35
5.6	Statistische Auswertung	36
6	Ergebnisse	37
6.1	Skalierbarkeit des Embedding-Vektors	37
6.2	Skalierbarkeit des Phi-Networks	37
6.3	Zusammenfassung	37
7	Diskussion	37
7.1	Interpretation der Ergebnisse	37
7.2	Implikationen für Swarm-RL	37
7.3	Vergleich mit ähnlichen Arbeiten	37
7.4	Grenzen	37
8	Fazit	37
8.1	Zusammenfassung	37
8.2	Beantwortung der Hypothesen	37
8.3	Zukünftige Arbeit	37

I Abkürzungsverzeichnis

AEC Agent Environment Cycle

RL Reinforcement Learning

MARL Multi-Agent Reinforcement Learning

KL-Constraint Kullback-Leibler Constraint

KL-Divergence Kullback-Leibler Divergence

CTDE Centralized Training Decentralized Execution

MLP Multi-Layer-Perceptron

GAE Generalized Advantage Estimation

GNN Graph Neural Networks

RBf Radial Basis Function

DNN Deep Neural Networks

MDP Markov Entscheidungsprozess

PPO Proximal Policy Optimization

MAPPO Multi Agent Proximal Policy Optimization

Dec-POMDP Decentralized Partially Observable Markov Decision Process

TRPO Trust Region Policy Optimization

VDN Value Decomposition Network

COMA Counterfactual Multi Agent

MADDPG Multi Agent Deep Deterministic Policy Gradient

MF-Q Mean-Field Q-Learning

MFAC Mean-Field Actor-Critic

RKHS Reproducing Kernel Hilbert Space

KME Kernel Mean Embeddings

MMD Maximum Mean Discrepancy

NTK Neural Tangent Kernel

PIC Permutation Invariant Critic

DGN Deep Graph Network

DQN Deep Q-Network

MAAC Multi-Actor-Attention-Critic

IB Information-Bottleneck

IBORM Information-Bottleneck-based Other agents' behavior Representation learning for Multi-agent reinforcement learning

GCN Graph Convolutional Network

SB3 Stable-Baselines3

IQM Interquartile Mean

1 Einleitung

Reinforcement Learning (RL) hat sich als effektives Werkzeug zur Lösung komplexer Entscheidungsprobleme in einer Vielzahl von Umgebungen etabliert, etwa in der Robotik, der Verkehrssteuerung oder der Spielstrategieoptimierung [1], [2]. Besonders Multi-Agent Reinforcement Learning (MARL), bei dem mehrere Agenten simultan lernen und interagieren, gilt als vielversprechender Ansatz zur Koordination von Schwarmsystemen – also Systemen, die aus vielen dezentralen Einheiten bestehen und durch lokale Interaktion ein emergentes globales Verhalten zeigen [3], [4].

Eine zentrale Herausforderung in MARL für Schwarmsysteme liegt in der hohen Dimensionalität der Zustands- und Beobachtungsräume, die sich aus der Vielzahl der Agenten und ihrer Interaktionen ergibt. Klassische Repräsentationsmethoden, etwa durch vollständige Zustandsvektoren oder histogrammbasierte Aggregationen, skalieren in solchen Szenarien nur unzureichend [5]. Kernel Mean Embeddings, implementiert durch neuronale Netze, stellen laut [6] eine vielversprechende Methode zur kompakten und informationsreichen Zustandsrepräsentation für hohe Agentenzahlen im RL dar. Allerdings handelt es sich bei dieser Methode um ein strukturelles Informations-Bottleneck, dessen Leistung durchaus von verschiedenen Faktoren abhängt. Daher bleibt bisher ungeklärt, inwieweit sich diese Repräsentation auch auf größere Schwarmgrößen übertragen lässt, ohne namhafte Einbußen bei Lernstabilität oder Leistung.

Ziel dieser Arbeit ist es daher, geeignete Metriken zur Bewertung der Skalierbarkeit von Mean-Embeddings im MARL-Kontext zu identifizieren, die Methode in moderne Frameworks für MARL zu integrieren und einen empirischen Beitrag zu leisten, indem erstmals systematisch die Skalierbarkeit von Mean-Embeddings untersucht wird. Der Begriff Skalierbarkeit umfasst hierbei die verschiedenen Dimensionen des Embedding-Vektors sowie die Tiefe und Breite des Phi-Netzwerks und wie sich diese auf verschiedene Trainings- und Test-Schwarmgrößen auswirken.

1.1 Motivation

Schwarmsysteme spielen in der Industrie eine immer größer werdende Rolle: Robotik, autonome Systeme oder auch verteilte Kontrolle sind nur ein paar der Gebiete, in denen sie eingesetzt werden [3], [4]. Sie sind robust, skalierbar, flexibel und dezentral [3]. Allerdings gehen auch einige Herausforderungen mit Schwärmen einher. So ist die technische Umsetzung oft anspruchsvoll, da das Verhalten zwar im Grunde einfach ist, aber dennoch ein genaues Modell und eine exakte Spezifikation nötig sind [3]. Zudem führen Eigenschaften wie Skalierbarkeit und Flexibilität vor allem bei großen Schwärmen häufig zu hochdimensionalen und verteilten Zustandsräumen, an denen einfache regelbasierte oder zentrale Steuerungen scheitern.

Ein natürlicher Schritt ist es dann, Machine-Learning-Algorithmen zu nutzen. Diese bieten den großen Vorteil, dass Agenten nicht händisch designt werden müssen, da das Verhalten der Agenten aus den Trainingsläufen hervorgeht. Weitergehend werden eine gute Adaptivität und Generalisierung bei richtig angewandtem Machine Learning gefördert [1]. Besonders bei mehreren identischen Agenten bietet sich MARL an. Es verfolgt das Prinzip der zentralisierten Lernphase bei dezentraler Ausführung, was es besonders geeignet für das Schwarm szenario macht, da die Erfahrungen aller Agenten in den Lernprozess einfließen [7].

Jedoch bringt auch MARL einige Herausforderungen mit sich. Eine der größten ist die Skalierbarkeit: Bei großen Schwärmen wachsen die Beobachtungsräume stark an, und variable Schwarmgrößen stellen zusätzliche Herausforderungen dar [5]. Hüttenrauch et al. zeigen, dass Mean Embeddings eine effektive Lösung für dieses Problem darstellen [6]. Dabei werden hochdimensionale Beobachtungen zu einer fixdimensionalen Repräsentation aggregiert, was kleinere Policy-Netzwerke erlaubt. Zudem bietet diese Methode die notwendige Permutations- und Skalierungsinvarianz, wodurch umliegende Agenten austauschbar sind und deren Anzahl variabel bleiben kann [6].

1.2 Forschungsfragen und Hypothesen

Diese Arbeit untersucht grundsätzlich die Repräsentationskapazität der Embedding-Vektoren und welchen Einfluss die Phi-Netzwerk Architektur hat. Es geht also um die Untersuchung von:

- Leistung
- Stabilität

- Generalisierung
- Skalierung

Um diese Aspekte gezielt zu untersuchen, werden folgende Forschungsfragen verfolgt:

- **RQ1: Einfluss der Embedding Dimension** Welchen Einfluss hat die Dimension des Embedding-Vektors auf Lernverhalten, Stabilität und Leistungsfähigkeit von [MARL](#)-Algorithmen in Schwarmsystemen unterschiedlicher Größe?
- **RQ2: Einfluss der ϕ -Netzwerk Architekturen**
Welche minimale Embedding-Dimension ist erforderlich, um eine vergleichbare Leistung zu einer höherdimensionalen Repräsentation bei gegebenen Schwarmgrößen zu erzielen?
- **RQ3: Minimale notwendige Repräsentationsdimension**
Wie skaliert die optimale Embedding-Dimension in Abhängigkeit von der Anzahl der Agenten im Schwarm, insbesondere beim Transfer von Trainings- zu größeren Test-Schwarmgrößen?
- **RQ4: Skalierung mit der Schwarmgröße**
In welchem Maß beeinflusst die Architektur das ϕ -Netzwerks die erforderliche Embedding-Dimension und die Skalierbarkeit der resultierenden Policy?

Basierend auf diesen Forschungsfragen, theoretischen Überlegungen und bisherigen empirischen Erkenntnissen werden folgende Hypothesen formuliert:

- **H1: Phase-Transition-Hypothese**
Die Leistungsfähigkeit von Mean-Embedding-basierten Policies bleibt für abnehmende Embedding-Dimensionen zunächst weitgehend stabil, bricht jedoch unterhalb einer aufgaben- und schwarmgrößenabhängigen kritischen Dimension stark ein.
- **H2: Sublineare Skalierung**
Die optimale Embedding-Dimension wächst sublinear mit der Anzahl der Agenten im Schwarm.
- **H3: Regularisierungseffekt kleiner Embeddings**
Kleinere Embedding-Dimensionen wirken als implizite Regularisierung und können die Generalisierungsfähigkeit auf größere Schwarmgrößen verbessern.
- **H4: Architekturabhängigkeit**
Tiefere oder breitere ϕ -Netzwerke reduzieren die minimal erforderliche Embedding-Dimension indem sie relevantere Merkmale extrahieren.

1.3 Ziel und Aufbau der Arbeit

Ziel dieser Arbeit ist die systematische und empirische Untersuchung der Skalierbarkeit von Mean-Embedding-basierten Repräsentationen im Kontext von [MARL](#) in Schwarmsystemen. Im Mittelpunkt steht die Analyse, inwieweit die Dimension des Mean-Embedding-Vektors sowie die Architektur des ϕ -Netzwerks die Leistungsfähigkeit, Lernstabilität und Generalisierungsfähigkeit der daraus resultierenden Policies beeinflussen.

Die Arbeit verfolgt einen rein empirischen Ansatz und zielt nicht auf die Entwicklung neuer Lernalgorithmen ab, sondern auf eine systematische Evaluation und Einordnung der Mean-Embedding-Architektur von Hüttenrauch et al. [6] im Hinblick auf ihre praktische Skalierbarkeit.

Die Arbeit ist wie folgt aufgebaut: [Abschnitt 2](#) führt die theoretischen Grundlagen ein und behandelt [MARL](#), Eigenschaften von Schwarmsystemen und skalierungsinvariante Repräsentationsmethoden. [Abschnitt 3](#) ordnet die Arbeit in den Kontext verwandter Ansätze zur Dimensionsreduktion und Aggregation in Multi-Agent-Systemen ein. ?? beschreibt die Implementierung der Mean-Embedding-Architektur, sowie die zugrunde liegenden Environments und Netzwerkstruktur. [Abschnitt 5](#) erläutert das experimentelle Design, die gewählten Metriken sowie die Konfigurationen der Experimente. [Abschnitt 6](#) präsentiert und analysiert die experimentellen Ergebnisse. [Abschnitt 7](#) diskutiert die Resultate im Hinblick auf Skalierbarkeit, Generalisierung und praktische Implikationen. Abschließend fasst [Abschnitt 8](#) die gewonnen Erkenntnisse zusammen und gibt einen Ausblick auf mögliche zukünftige Arbeiten.

2 Hintergrund

2.1 Reinforcement Learning: Grundlagen

Reinforcement Learning bezeichnet das Lernen möglichst optimaler Strategien zum Lösen spezifischer Probleme durch Interaktion eines Agenten mit seiner Umgebung [1]. Im Folgenden werden die für die Arbeit relevanten Grundlagen eingeführt, beginnend bei der Modellierung des Problems als Markov-Entscheidungsprozess bis hin zu modernen Policy-Gradient Verfahren, auf denen die Algorithmen Proximal Policy Optimization (PPO) und Trust Region Policy Optimization (TRPO) aufbauen.

2.1.1 Markov Entscheidungsprozess

Das Standardmodell für sequenzielle Entscheidungsprozesse im Reinforcement Learning ist der Markov Entscheidungsprozess (MDP) [1, Kap.3]. Ein MDP ist als Tupel $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ definiert, wobei $\mathcal{S}$ den Zustandsraum und $\mathcal{A}$ den Aktionsraum bezeichnet. Die Transitionsfunktion $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ beschreibt die Wahrscheinlichkeit vom Zustand f durch Aktion a vom Zustand f in den Folgezustand f' zu überzugehen. Die Rewardfunktion $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ weist dabei jeder Kombination aus Zustand und Aktion eine skalare Belohnung zu. Der Discount-Faktor $\gamma \in [0, 1]$ gewichtet zukünftige Belohnungen relativ zu unmittelbaren. Charakteristisch für ein MDP ist die Markov-Eigenschaft, nach der die zukünftige Zustandsentwicklung ausschließlich vom aktuellen Zustand und der gewählten Aktion abhängt [1, Gl. 3.8]:

$$P(s_{t+1} | s_t, a_t) = P(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0). \quad (1)$$

Diese Eigenschaft erlaubt es, die Historie zu einem einzigen Zustand zu komprimieren, ohne Informationen für die optimale Entscheidung zu verlieren. Ein trajectory $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ entsteht durch die wiederholte Interaktion des Agenten mit seiner Umgebung. Der discounted return ab Zeitschritt t ist dabei definiert als

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \quad (2)$$

[1, Gl. 3.2] und stellt die zentrale Größe dar, die ein Agent durch geeignete Aktionswahl zu maximieren versucht.

2.1.2 Policy und Valuefunktionen

Eine Policy π beschreibt die Strategie eines Agenten zu Auswahl der Aktionen in einem gegebenen Zustand und ist als bedingte Wahrscheinlichkeitsverteilung

$$\pi(a | s) = P\{A_t = a | S_t = s\} \quad (3)$$

über den Aktionsraum definiert [1, Kap. 3.1]. Im Kontext von Deep Reinforcement Learning wird die Policy üblicherweise durch ein neuronales Netz π_θ mit den Parametern θ repräsentiert. Um die Qualität einer Policy quantifizieren zu können, werden zwei eng verwandte Funktionen definiert. Die *State Value Function* $v_\pi(s)$ beschreibt den unter π erwarteten Return ausgehend vom Zustand s :

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s \right]. \quad (4)$$

Die *Action Value Function* $q_\pi(s, a)$ erweitert diese Definition um eine konkrete Aktion a im Zustand s :

$$q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t | S_t = s, A_t = a]. \quad (5)$$

Beide Funktionen sind grundlegend für nahezu alle Reinforcement Learning Algorithmen, da sie die Bewertung von Zuständen und Zustands-Aktions Paaren ermöglichen [1, Kap. 3.7].

Eine der zentralen Eigenschaften einer Value Function ist ihre rekursive Struktur, die durch die *Bellman Erwartungsgleichung* ausgedrückt wird:

$$v_\pi(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')]. \quad (6)$$

Sie zerlegt den Wert eines Zustands in die unmittelbar erwartete Belohnung und den discounted Value des Folgezustands und stellt damit die Grundlage iterativer Lösungsverfahren wie z.B. die Policy Evaluation und Value Iteration dar [1, Kap. 3.7].

Eine Policy π^* wird optimal genannt, wenn ihre Zustandsfunktion $v_{\pi^*}(s) = v_*(s)$ für alle $s \in \mathcal{S}$ den Wert jeder anderen Policy erreicht oder übertrifft. Sie erfüllt die *Bellman Optimalitätsgleichung*, in der die Erwartung über π durch ein Maximum über die Aktionen ersetzt wird [1, Kap. 3.8]. Leider ist die direkte Lösung dieser Gleichung nur für sehr kleine, vollständig bekannte MDPs möglich. In der Praxis müssen die Value Functions approximativ aus Stichproben gelernt werden.

2.1.3 Optimalitätsziel

Das Lernziel im Reinforcement Learning besteht darin, eine Policy π^* zu finden, welche den erwarteten kumulativen Return über alle Zustände maximiert. Über Value Functions lassen sich verschiedene Policies miteinander vergleichen. Eine Policy π ist mindestens so gut wie π' , wenn ihr erwarteter Return in jedem Zustand mindestens genau so hoch ist, also $v_\pi(s) \geq v_{\pi'}(s)$ für alle Zustände s gilt [1, Kap. 3.8]. Es lässt sich zeigen, dass für jeden endlichen MDP mindestens eine Policy existiert, die in diesem Sinne optimal ist. Alle solchen Policies teilen sich dieselbe optimale State Value Function

$$v_*(s) \doteq \max_{\pi} v_{\pi}(s), \quad \forall s \in \mathcal{S} \quad (7)$$

und die optimale Action Value Function

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a) \quad (8)$$

Im Kontext parametrisierter Policies wird das Optimierungsziel äquivalent als Maximierung der Performance Metrik

$$J(\theta) \doteq \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right] \quad (9)$$

über die Policy Parameter θ formuliert [8]. Die Erwartung wird dabei über alle Trajectories τ gebildet, die unter der Policy π_{θ} durch Interaktionen mit der Umgebung entstehen. Dieses Optimierungsproblem wird durch zwei unterschiedliche Ansätze gelöst.

Wertbasierte Verfahren approximieren die optimale Action Value Function q_* durch einen Funktionsapproximator und leiten die Policy implizit durch greedy Auswahl der gelernten Werte ab. Ein sehr bekanntes Beispiel ist das Deep Q Network [2], welches in zahlreichen Atari-Spielen menschliches Niveau erreicht hat.

Policy basierte Verfahren parametrisieren hingegen die Policy π_{θ} direkt und optimieren $J(\theta)$ durch gradientenbasierte Verfahren mit einem geeigneten Gradienten der Performance Metrik [1, Kap. 9.2].

Für die in dieser Arbeit betrachteten Schwarmsysteme sind policy basierte Verfahren aus verschiedenen Gründen vorzuziehen: Sie skalieren besser auf hochdimensionale, kontinuierliche Aktionsräume, liefern stochastische Policies welche Exploration und Robustheit gegenüber gleichzeitig handelnden Agenten begünstigen, und sie sind methodisch direkt mit den in 2.2 eingeführten Trust Region Verfahren kompatibel. Die Berechnung des Gradienten von $J(\theta)$ im Bezug auf die Policy-Parameter behandelt der folgende Abschnitt.

2.1.4 Policy Gradient Theorem

Die direkte Berechnung des Gradienten der Performance-Metrik $J(\theta)$ nach den Policy-Parametern θ erweist sich als problematisch: $J(\theta)$ hängt nicht nur über die Policy π_{θ} vom Parametervektor ab, sondern auch über die durch π_{θ} induzierte stationäre Zustandsverteilung. Diese Verteilung ist im Allgemeinen unbekannt, da sie von der Dynamik der Umgebung abhängt. Das *Policy Gradient Theorem* [8] liefert eine Darstellung des Gradienten, in der die Zustandsverteilung implizit über Stichprobentrajektorien einfließt und somit nicht explizit modelliert werden muss.

Für eine differenzierbare parametrisierte Policy π_{θ} gilt:

$$\nabla_{\theta} J(\theta) \propto \sum_s \mu_{\pi}(s) \sum_a q_{\pi}(s, a) \nabla_{\theta} \pi_{\theta}(a | s), \quad (10)$$

wobei μ_π die unter π_θ induzierte on-policy Zustandsverteilung bezeichnet [1, Gl.]. Durch den Einsatz des *Log-Likelihood Tricks* $\nabla_\theta \pi_\theta(a | s) = \pi_\theta(a | s) \nabla_\theta \log \pi_\theta(a | s)$ lässt sich die Gleichung (10) äquivalent als Erwartungswert unter der Policy schreiben:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(A_t | S_t) q_{\pi_\theta}(S_t, A_t)]. \quad (11)$$

Diese Formulierung ist der Schlüssel zu der praktischen Anwendung von Policy-Gradienten. Dieser kann nun durch Stichprobenmittelwerte aus Trajektorien geschätzt werden, ohne dass die Umgebungsdynamik explizit gekannt werden muss. Konkret bedeutet dies, dass genau die Aktionen wahrscheinlicher gemacht werden, die zu hohen erwarteten Returns geführt haben, und solche mit niedrigen Returns unwahrscheinlicher.

Die Grundlage einer breiten Klasse an Algorithmen die auf diesem Konzept aufbauen bildet REINFORCE [9]. Williams führt diese ursprünglich für verbundene Netze mit stochastischen Einheiten ein. In der heute verbreiteten, für episodische MDPs spezialisierten Form [1, Kap. 13.3] wird der unbekannte Aktionswert in der Gleichung (11) durch den tatsächlich beobachteten Return G_t aus einer vollständigen Trajektorie ersetzt. Die daraus resultierende Update-Regel

$$\theta_{t+1} \doteq \theta_t + \alpha G_t \frac{\nabla \pi(A_t | S_t, \theta_t)}{\pi(A_t | S_t, \theta_t)} \quad (12)$$

liefert eine unverzerrte Schätzung des Policy-Gradienten [1, Gl. 13.8]. Der Bruch $\nabla \pi / \pi$, auch als *Eligibility Vector* bezeichnet [1, Kap. 13.3] ist äquivalent zu $\nabla \ln \pi(A_t | S_t, \theta_t)$ und bildet die einzige Stelle, an der die Policy Parametrisierung in das Update miteingeht.

Trotz dieser doch sehr eleganten Lösung weisen REINFORCE-Verfahren als Monte-Carlo Methoden eine hohe Varianz des geschätzten Gradienten auf, da G_t über die gesamte verbleibende Trajektorie immer summiert wird und somit auch von zufälligen Ereignissen weit in der Zukunft beeinflusst wird. Das führt in der Praxis zu einer langsamen und instabilen Konvergenz und motiviert den Einsatz von Value-Functions als Baseline, wie sie im Folgenden Abschnitt im Rahmen der Actor-Critic Verfahren behandelt werden.

2.1.5 Actor-Critic Architektur

Die hohe Varianz bei reinen Monte-Carlo Verfahren wie REINFORCE motiviert die Einführung einer Baseline, einer von der Aktion unabhängigen Vergleichsgröße $b(s)$, die vom Return subtrahiert wird [1, Kap. 13.4]. Das verallgemeinerte Policy-Gradient Theorem

$$\nabla J(\theta) \propto \sum_s \mu(s) \sum_a (q_\pi(s, a) - b(s)) \nabla \pi(a | s, \theta) \quad (13)$$

bleibt unter dieser Modifikation noch immer gültig, da der zusätzliche Term im Erwartungswert Verschwindet: $\sum_a b(s) \nabla \pi(a | s, \theta) = b(s) \nabla \sum_a \pi(a | s, \theta) = b(s) \nabla 1 = 0$. Die Baseline lässt den Erwartungswert des Gradienten somit unverändert, kann allerdings dessen Varianz erheblich reduzieren [1, Kap. 13.4].

Eine gute Wahl für die Baseline ist eine Schätzung der Zustandswertfunktion $\hat{v}(s, \mathbf{w})$ mit dem Gewichtsvektor $\mathbf{w}$. In Actor-Critic Verfahren wird diese gelernte Wertfunktion nicht nur als Baseline, sondern auch zum *Bootstrapping* verwendet. Der vollständige Monte-Carlo Return G_t wird durch den One-Step Return $G_{t:t+1} = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$ ersetzt [1, Kap.13.5]. Das update reduziert sich damit auf

$$\theta_{t+1} \doteq \theta_t + \alpha \delta_t \frac{\nabla \pi(A_t | S_t, \theta_t)}{\pi(A_t | S_t, \theta_t)} \quad (14)$$

mit dem *TD-Error*

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w}), \quad (15)$$

der die Differenz zwischen dem unmittelbar nach einem Schritt verfügbaren, gebootstrappten Schätzwert $R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$ und der aktuellen Schätzung des values $\hat{v}(S_t, \mathbf{w})$ misst und in Sutton und Bartos Werk als die zentrale Größe Temporal-Difference basierter Verfahren eingeführt wird [1, Kap. 6].

Die dadurch entstehende Architektur besteht aus zwei gemeinsam trainierten Komponenten. Dem *Actor* π_θ , der Aktionen auswählt, und dem *Critic* $\hat{v}_\mathbf{w}$, der durch Bewertung des Folgezustands die

Qualität der Aktion einschätzt [1, Kap. 13.5]. Diese Konstruktion ist die historisch gesehen erste, in der eine gelernte Value Function explizit zur Bewertung gewählter Aktionen eingesetzt wird [10].

Gegenüber REINFORCE Algorithmen bringt der Übergang zum One-Step Return einen Bias-Varianz Tradeoff mit sich. Die Varianz des geschätzten Gradienten-Estimators wird durch das Bootstrapping deutlich reduziert. Gleichzeitig führt die Approximation $\hat{v}_w \neq v_\pi$ allerdings einen Bias ein. In der Praxis wird das einfache δ_t aus Gleichung (15) daher selten direkt als Advantage-Estimator verwendet. Die Generalized Advantage Estimation (GAE) von [11] verallgemeinert diesen Estimator als exponentiell gewichtete Summe von TD-Residuen δ_t

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad (16)$$

und führt einen zweiten Parameter $\lambda \in [0, 1]$ ein, mithilfe dessen sich der Bias-Varianz Kompromiss explizit steuern lässt. $\lambda = 0$ reduziert den Estimator auf das TD-Residuum, $\lambda = 1$ hingegen entspricht dem unverzerrten Monte-Carlo Advantage. GAE steht damit Analog zu in $\text{TD}(\lambda)$ verwendeten Eligibility-Trace Konstruktion auf der Eben des Advantage-Estimators und ist heute der standard Estimator in modernen On-Policy Verfahren.

In Deep Reinforcement Learning werden π_θ und $\hat{v}_w$ typischerweise als neuronale Netze mit gemeinsamen oder getrennten Parametern implementiert [12]. Diese Actor-Critic Architekturstur in Kombination mit GAE bildet die Grundlage der in Abschnitt 2.2 behandelten Trust-Region Verfahren, in welchen zusätzlich eine Begrenzung der Änderungen in der Policy pro Update eingeführt wird, um problematische, große Updates zu verhindern. Die konkrete Implementierung dieser Algorithmen in der in dieser Arbeit verwendeten Bibliothek `stable-baselines3` nutzt GAE als Default Advantage Estimator [13]

2.2 Deep Reinforcement Learning: TRPO und PPO

Die in Abschnitt 2.1 vorgestellten Policy Gradient und Actor-Critic Verfahren leiden allerdings an einer zentralen Schwäche. Die Update-Regeln basieren nämlich auf einer lokalen linearen Approximation der Performance-Metrik $J(\theta)$ um den aktuellen Parametervektor herum, weshalb zu große Schritte die Policy in Bereiche des Parameterraums führen können, in denen diese Approximation ungültig ist. Als Folge kann ein einziges, zu großes Update eine zuvor gut trainierte Policy kollabieren lassen. In der Literatur ist dies als *Policy Collapse* bekannt und resultiert aus einer hohen Varianz der Gradienten-Estimators kombiniert mit einer fest gewählten Lernrate. Da $J(\theta)$ über Trajectories definiert ist, kann auch eine geringfügige Änderung der Policy zu drastisch veränderten Zustandsverteilungen führen, sodass die Aufnahme einer lokalen Linearität besonders schnell verletzt wird [14].

Trust-Region Verfahren adressieren dieses Problem durch eine explizite Begrenzung der Policy-Änderung pro Update. Anstelle eines unbeschränkten Gradientenaufstiegs wird die Optimierung auf eine *Vertrauensregion*, die Trust-Region, um die aktuelle Policy beschränkt, innerhalb derer die Surrogate-Approximation verlässliche Vorhersagen über die tatsächliche Performance-Verbesserung liefert. Die im Folgenden behandelten Algorithmen TRPO [14], welchen Hüttenrauch et al. in ihrem Paper nutzen und PPO [15], welcher heutzutage oft Standard ist, realisieren dieses Prinzip auf unterschiedliche Weise. TRPO formuliert die Trust-Region als harten Constraint über die Kullback-Leibler Divergence (KL-Divergence) zweier aufeinander folgender Policies und garantiert auf dieser Basis monotone Verbesserung, während PPO dieselbe Idee durch ein einfacheres, clipped Surrogate-Objective approximiert und so mit standard Optimierungsverfahren kompatibel wird. Vor allem PPO bildet heute einen de-facto standard im On-Policy Deep Reinforcement Learning.

2.2.1 Trust Region Policy Optimization

Ausgangspunkt von TRPO ist eine exakte Zerlegung der Performance-Differenz zwischen der aktuellen Policy $\pi_{\theta_{old}}$ und einer Kandidaten-Policy π_θ in eine über die Zustandsverteilung unter π_θ gemittelte Erwartung der Advantage-Funktion von $\pi_{\theta_{old}}$ [14, Gl. 1]. Da diese Zustandsverteilung selbst von π_θ abhängt und somit erst nach der Aktualisierung verfügbar wäre, führt [14] ein *Surrogate-Objective* ein, in dem die unbekannte Zustandsverteilung durch die bereits beobachtete Verteilung unter $\pi_{\theta_{old}}$ ersetzt wird

$$L_{\theta_{old}}(\theta) = \mathbb{E}_{s \sim p_{\theta_{old}}, a \sim \pi_{\theta_{old}}} \left[\frac{\pi_\theta(a | s)}{\pi_{\theta_{old}}(a | s)} A^{\pi_{\theta_{old}}}(s, a) \right]. \quad (17)$$

Das Verhältnis $\pi_\theta(a | s)/\pi_{\theta_{old}}(a | s)$ ist das aus dem Importance Sampling bekannte Likelihood-Verhältnis und ermöglicht es, den Erwartungswert über Trajectories der alten Policy zu berechnen obwohl der zu bewertende Term von der neuen Policy abhängt [14].

Eine zentrale theoretische Aussage aus **TRPO** ist die monotone Verbesserungsgarantie [14, Thm. 1]. Die tatsächliche Verbesserung der Performance $\eta(\pi_\theta) - \eta(\pi_{\theta_{old}})$ ist nach unten durch das Surrogate-Objective abzüglich eines Strafterms, der proportional zur maximalen **KL-Divergence** zwischen beiden Policies ist, beschränkt. Aus dieser unteren Schranke folgt, dass jede Aktualisierung, welche die rechte Seite verbessert, auch die tatsächliche Performance verbessert und damit ein monoton verbesserndes Verfahren existiert. Da der Strafterm allerdings in der Praxis jedoch zu konservative Updates erzwingt, ersetzt **TRPO** ihn durch einen Kullback-Leibler Constraint (**KL-Constraint**) und löst stattdessen das constrained Optimierungsproblem

$$\max_{\theta} L_{\theta_{old}}(\theta) \quad \text{u.B.v.} \quad \bar{D}_{KL}(\theta_{old}, \theta) \leq \delta, \quad (18)$$

wobei $\bar{D}_{KL}$ die unter $\rho_{\theta_{old}}$ gemittelte **KL-Divergence** und δ die Größe der Trust-Region darstellt [14, Gl. 12]. Üblich ist $\delta \in [10^{-3}, 10^{-2}]$.

Die praktische Lösung von Gleichung (18) geschieht in zwei Schritten. Zuerst wird das Optimierungsproblem durch eine lineare Approximation des Surrogate-Objectives und eine quadratische Approximation des KL-Constraints lokal vereinfacht, was zu einer Suchrichtung führt, die formal dem natürlichen Policy-Gradient entspricht [16]. Diese Suchrichtung wird durch das *Conjugate Gradient* berechnet, ohne die Fisher-Informationsmatrix explizit zu invertieren [14, Abs. 6]. Anschließend stellt ein *Backtracking Line Search* sicher, dass der tatsächliche Update-Schritt sowohl den **KL-Constraint** einhält, als eben auch das unapproximierte Surrogate-Objective erhöht. Anderenfalls wird die Schrittweite exponentiell reduziert.

TRPO liefert damit ein theoretisch fundiertes On-Policy Verfahren, mit empirisch robuster Performance auf hochdimensionalen Continuous-Control- und Atari-Benchmarks [14]. Die Hauptkritikpunkte aus praktischer Sicht sind:

- Hohe Rechenkosten pro update, bedingt durch die Hessian Vektor Produkte des Conjugate-Gradient Solvers
- Nicht-Parallelisierbares Line-Search
- Beschränkung auf einen einzigen Gradientenschritt pro gesammeltem Rollout, was zu einer geringen Sample-Effizienz führt

Diese Limitierungen motivieren die im folgenden Abschnitt behandelte Vereinfachung durch **PPO**.

2.2.2 Proximal Policy Optimization

PPO [15] verfolgt dieselbe Grundidee wie **TRPO**, die Beschränkung der Policy-Änderung pro Update, ersetzt jedoch den harten **KL-Constraint** durch eine einfachere, mit First-Order Methoden kompatible Konstruktion. Ausgangspunkt ist dieselbe Importance-Sampling basierte Surrogate-Form wie in Gleichung (17), welche in [15] kompakt über das Likelihood-Verhältnis

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \quad (19)$$

ausgedrückt werden kann. Das ungeklippte Surrogate Objective $L^{CPI}(\theta) = \hat{\mathbb{E}}_t[r_t(\theta) \hat{A}_t]$ würde ohne eine Beschränkung große Policy Updates erlauben, sobald das Verhältnis r_t stark von 1 abweicht [15, Abschn. 3]. Um dieses Problem zu vermeiden, ohne auf einen expliziten Constraint zurückzugreifen, schlägt [15] ein *Clipped Surrogate Objective* vor

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right], \quad (20)$$

wobei $\hat{A}_t$ ein Advantage Estimator ist. Typischerweise wird dafür der **GAE** aus (16). ϵ gibt den Clipping Radius an, für welchen im Originalpaper der Wert $\epsilon = 0,2$ empfohlen wird. Der Clipping Operator beschränkt $r_t(\theta)$ auf das Intervall $[1 - \epsilon, 1 + \epsilon]$, und das Minimum stellt sicher, dass L^{CLIP} eine

pessimistische untere Schranke des ungeklippten Surrogate-Objectives bildet. In der Konsequenz wird ein Update ignoriert, sobald es die Policy in einer Richtung ändert, die durch das Clipping bereits eingegrenzt wurde. Bei positivem $\hat{A}_t$ verhindert die obere Schranke $1 + \epsilon$ weitere Erhöhungen von r_t , bei negativem $\hat{A}_t$ verhindert die untere Schranke $1 - \epsilon$ weitere Verringerungen.

Diese mathematische Konstruktion erlaubt es, das Surrogate-Objective über mehrere Epochen mit standard Gradienten Methoden auf dem gesammelten Rollout zu optimieren, ohne dass die Policy sich unkontrolliert weit vom Sampling-Verhalten entfernt. Bei **TRPO** hingegen ist je Iteration nur ein einzelner Gradientenschritt zulässig. Der vollständige PPO-Verlust kombiniert das Clipped Objective mit einem Value-Function-Loss und einem optionalen Entropy-Bonus um die Exploration zu fördern [15, Abschn. 5].

$$L^{\text{PPO}}(\theta) = \hat{\mathbb{E}}_t \left[L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 \mathcal{H}[\pi_\theta](s_t) \right], \quad (21)$$

mit den Koeffizienten c_1, c_2 und $L^{\text{VF}} = (\hat{v}_w(s_t) - V_t^{\text{target}})^2$.

Bei empirischen Beobachtungen auf standard Benchmarks erreicht **PPO** eine vergleichbare oder bessere Performance als **TRPO** bei deutlich geringerem Implementierungs- und Rechenaufwand und ist heute der de-facto Standard im On-Policy Deep Reinforcement Learning [15]. Spätere Systematische Untersuchungen [17], [18] haben allerdings gezeigt, dass die in der Praxis erreichbare Performance maßgeblich von Code-Level Optimierungen wie Observation- und Advantage-Normalisierung, orthogonaler Gewichtsinitialisierung sowie Lernraten-Annealing abhängt. Dies ist ein Aspekt, der in Abschnitt 5 bei der Diskussion der Wahl der konkreten Hyperparameter wieder aufgegriffen wird.

2.3 Multi Agent Reinforcement Learning

Die durch den vorherigen Abschnitt behandelten Verfahren gehen durchweg von einem einzelnen Agenten aus, welcher mit einer stationären Umgebung interagiert. In Anwendungen wie der Schwarmrobotik, Verkehrssteuerung, Mehrspielerspiele und kollaborativen Robotik agieren jedoch mehrere lernende Entitäten gleichzeitig in einer gemeinsamen Umgebung und beeinflussen sich gegenseitig durch ihre Aktionen. **MARL** befasst sich mit der Erweiterung des klassischen, auf nur einen Agenten ausgelegten Reinforcement Learning Paradigmas auf eben solche System [19], [20].

Aus der Perspektive eines einzelnen Agenten verlieren mehrere zentrale Annahmen des **MDP**-Formalismus in **MARL** Settings ihre Gültigkeit. Da jeder Agent die durch die anderen Agenten induzierten Dynamiken als Teile seiner eigenen Umgebung wahrnimmt, ist diese Umgebung aus seiner Sicht *nicht stationär*. Während des Lernens verändern sich die Policies der Mitspielenden, sodass die effektive Übergangs- und Belohnungsdynamik nicht mehr zeitlich konstant ist. Zudem führt das gemeinsame Handeln mehrerer Agenten zu einem zusammengekommenen (Joint) Action Space, dessen Kardinalität exponentiell mit der Anzahl der Agenten mitwächst. Bekannt ist dieses Skalierungsproblem in der Literatur dazu als *Curse of Dimensionality*, das klassische tabellarische und auch viele Deep Reinforcement Learning Verfahren in der direkten Anwendung leider untauglich macht [5].

Im Folgenden werden die für diese Arbeit relevanten Grundlagen von **MARL** in vier Schritten eingeführt. 2.3.1 führt die formalen Modelle der Markov Games und Decentralized Partially Observable Markov Decision Process (**Dec-POMDP**)s ein, mit denen sich Multi Agent Systeme beschreiben lassen. 2.3.2 diskutiert die zentralen Herausforderungen Nicht-Stationarität, Credit Assignment und Skalierung. 2.3.3 stellt mit dem Paradigma Centralized Training Decentralized Execution (**CTDE**) sowie Parameter Sharing zwei strukturelle Lösungsansätze vor und führt mit Multi Agent Proximal Policy Optimization (**MAPPO**) [21] die für diese Arbeit relevante Multi-Agent Variante von **PPO** ein. 2.3.4 schlägt schließlich über Mean-Field **MARL** [22] die Brücke zu Schwarmsystemen, in denen die schiere Zahl der Agenten zusätzliche strukturelle Annahmen erforderlich macht und das Konzept der permutationsinvarianten Aggregation motiviert, das später Gegenstand von 2.4 und 2.5 ist.

2.3.1 Markov Games und Dec-POMDP

Markov Games sind eine formale Erweiterung von **MDPs** auf mehrere Agenten und gehen auf [23] zurück, der das Konzept des *Markov Game* (teils auch *stochastic game* genannt) als Rahmen für **MARL** eingeführt hat. Ein Markov Game mit N Agenten ist definiert als ein Tupel $\langle S, A_i \rangle_{i=1}^N, p, \{r_i\}_{i=1}^N, \gamma \rangle$ wobei S den gemeinsamen Zustandsraum und A_i den Aktionsraum des Agenten i bezeichnet. Die

$$p(s' | s, a_1, \dots, a_N) \quad (22)$$

hängt vom aktuellen Zustand und dem gemeinsamen Aktionsvektor aller Agenten ab, und jeder Agent i erhält eine individuelle Belohnung $r_i : S \times A_1 \times \dots \times A_N \rightarrow R$. Während die Originalformulierung von [23] sich auf den Fall von zwei Agenten und zero-Sum Spiele konzentriert und mit dem Minimax-Q Algorithmus eine Lösung für genau diese Klasse vorschlägt, ist die Verallgemeinerung auf N Agenten und eine allgemeine Reward Funktion in der Literatur Standard [19, Kap. 3]. Im völlig kooperativen Spiel teilen alle Agenten dieselbe Reward-Funktion $r_1 = r_2 = \dots = r_N$, da alle Agenten versuchen, ein gemeinsames Ziel zu erreichen. Diese Art des Markov Games beschreibt das in dieser Arbeit betrachtete Schwarm-Szenario.

In realistischen MARL-Systemen verfügt jedoch kein einzelner Agent über die vollständigen Informationen des globalen Zustandes. [24] formalisieren diese Situation als **Dec-POMDP**, eine Erweiterung des klassischen POMDP auf mehrere Agenten und auch des Markov Game auf nur teilweise beobachtbare Observationsräume. Ein **Dec-POMDP** wird durch das Tupel $\langle S, \{\mathcal{A}_i\}, \{\mathcal{O}_i\}, p, O, r, \gamma \rangle$ beschrieben, in dem jedem Agenten i ein eigener Beobachtungsraum $\mathcal{O}_i$ zugeordnet ist. Die Beobachtungsfunktion $O(o_1, \dots, o_N | a_1, \dots, a_N)$ gibt die Wahrscheinlichkeit an, mit der die Agenten nach Aktion und Zustandsübergang ihre jeweiligen individuellen Beobachtungen erhalten. Im kooperativen **Dec-POMDP** teilen die Agenten eine gemeinsame Reward Funktion r , wobei das Optimierungsziel darin besteht, eine *joint policy* $\pi = \pi_1, \dots, \pi_N$ zu finden, die den erwarteten gemeinsamen Return maximiert. Jede individuelle Policy π_i darf dabei allerdings nur auf die lokal verfügbare Beobachtungshistorie des Agenten i zurückgreifen.

Ein zentrales Resultat zu diesen speziellen Markov Games ist die Berechnungskomplexität. Bernstein, Zilberstein und Immerman [24] zeigen, dass das endliche **Dec-POMDP** Problem bereits für $N = 2$ Agenten *NEXP-vollständig* ist und unter der Annahme $\text{EXP} \neq \text{NEXP}$ keine Lösung in Polynomialzeit zulässt, wodurch sich **Dec-POMDPs** fundamental von zentralisierten POMDPs, welche im Worst Case lediglich eine exponentielle Laufzeit erfordern, unterscheidet. Diese stark erhöhte Komplexität impliziert, dass eine optimale dezentrale Lösung für Mehragentensysteme nicht in vertretbarer Zeit berechnet werden kann. Dies macht die Suche nach *approximativen Verfahren* und insbesondere nach strukturellen Annahmen, wie die später erwähnten permutationsinvarianten Beobachtungsaggregationen 2.5, umso relevanter.

In der Implementierung dieser Arbeit wird das von [25] vorgeschlagene Agent Environment Cycle (**AEC**) Modell der MARL-Bibliothek **PettingZoo** verwendet, in dem die Agenten sequenziell agieren und nach jeder Aktion eine Umgebungsantwort erhalten. [25] beweisen die Äquivalenz zwischen **AEC**-Spielen und partially observable markov games und zeigen damit, dass die sequenzielle Reformulierung die Ausdruckskraft der **Dec-POMDPs** weiter aufrechterhält allerdings dabei typische Implementierungsfehler bei simultaner Aktionswahl vermeidet.

2.3.2 Herausforderungen von MARL

Die Komplexität von **Dec-POMDPs** spiegelt sich in einer Reihe praktischer Herausforderungen, die bei der Anwendung von **MARL** auftreten. Drei dieser Herausforderungen sind für die in dieser Arbeit betrachteten Arbeit besonders relevant.

Nicht Stationarität Aus der Perspektive eines einzelnen Agenten i wird die Umgebungs-dynamik durch die Policies aller anderen Agenten π_{-i} mitbestimmt. Während des Trainings verändern sich diese Policies kontinuierlich, sodass die effektive Übergangsverteilung aus Sicht des Agenten i

$$p_i(s' | s, a_i) = \sum_{a_{-i}} \pi_{-i}(a_{-i} | s) p(s' | s, a_i, a_{-i}) \quad (23)$$

nicht zeitlich konstant ist [5]. Damit verliert der zentrale Konvergenznachweis für die Single-Agent Verfahren seine Gültigkeit, da dieser auf einer stationären Markov-Umgebung beruht. Insbesondere off-policy Algorithmen wie Q-Learning, die auf einem Experience Replay [2] basieren, sind betroffen. Gespeicherte Übergänge stammen aus früheren Policies der Mitspielenden und spiegeln die aktuelle Dynamik aufgrund der geupdateten Policies nicht mehr wieder [7]. Diese Beobachtung ist eine der zentralen Motivationen für die in Abschnitt 2.3.3 behandelten Ansätze, in denen der Zugriff des Critics auf die Aktionen der Policies der anderen Agenten erhält.

Multi-Agent Credit Assignment In kooperativen Szenarien teilen sich alle Agenten eine gemeinsame Reward-Funktion. aus der beobachteten kollektiven Belohnung lässt sich jedoch nicht ohne Weiteres ableiten, welcher Agent durch welche Aktion zum Erfolg oder Misserfolg beigetragen hat [19, Kap. 5]. Diese Schwierigkeit der *Beitragszuordnung* ist im Single-Agent Fall nicht existent und wird in der MARL Literatur als *multi-agent credit assignment problem* bezeichnet. Klassische Lösungsansätze reichen von value-decomposition Verfahren, wie Value Decomposition Network (VDN) [26] und QMIX [27], die die globale Q-Funktion faktorisieren, bis hin zu counterfactual baselines wie in Counterfactual Multi Agent (COMA) [28], die für jeden Agenten den marginalen Beitrag zur Gesamtbelohnung schätzen. In der vorliegenden Arbeit wird das Problem entsprechend der Vorgehensweise von [6] durch eine vollständige Parametrisierung mit *Parameter Sharing* adressiert, wobei alle Agenten dieselbe Policy ausführen und damit dasselbe Gradienten-Signal erhalten.

Skalierung mit der Agentenzahl Mit zunehmender Agentenzahl N wachsen sowohl der Joint-Action-Space $A_1 \times \dots \times A_N$ als auch der Beobachtungsraum exponentiell, bzw. linear, sodass die Eingabedimension auf allen Beobachtungen operierender Policies extrem anwächst. Selbst bei moderater Anzahl von wenigen Dutzend Agenten führt dies zu einer hohen Trainingsvarianz und einem unverhältnismäßig schnellen Anstieg der Sample-Komplexität [5], [20]. Klassische MARL Algorithmen wie Multi Agent Deep Deterministic Policy Gradient (MADDPG) [7] sind aus diesem Grund in ihrer Originalformulierung auf kleine Agentenzahlen ($N \lesssim 10$) beschränkt. In Schwarmssysteme, die typischerweise mehrere zehn bis hunderte Agenten umfassen, ist diese Skalierungsgrenze besonders einschränkend und motiviert strukturelle Annahmen wie Permutationsinvarianz und Homogenität der Agenten. Diese Annahmen erlauben eine Aggregation der Beobachtungen mit konstanter Eingabedimension, unabhängig von N . Diese Idee wird in 2.3.4 und noch einmal ausführlich in 2.5 aufgegriffen.

2.3.3 Central Training Decentralized Execution und Parameter Sharing

Die bereits identifizierten Herausforderungen werden in der modernen Forschung zu MARL durch zwei komplementäre strukturelle Konzepte adressiert: das CTDE Paradigma und das *Parameter Sharing* unter homogenen Agenten. Beide Konzepte stehen orthogonal zueinander und werden in der vorliegenden Arbeit gemeinsam eingesetzt.

Centralized Training, Decentralized Execution Das CTDE Paradigma wurde von Lowe, Wu, Tamar u. a. [7] im Kontext von MADDPG eingeführt und ist seither der dominante Designansatz im Deep MARL. Die Grundidee ist, dass während der Trainingsphase Informationen, die zur Ausführungszeit nicht verfügbar sind, zu nutzen. Typischerweise den globalen Zustand oder die Beobachtungen und Aktionen aller Agenten. Konkret wird in MADDPG ein gemeinsamer Critic

$$Q_i^\pi(s, a_1, \dots, a_N) \quad (24)$$

trainiert, der für Agent i Zugriff auf den globalen Zustand und die gemeinsamen Aktionen aller Agenten hat. Die einzelnen Actors $\pi_i(a_i | o_i)$ hingegen werden hingegen meist nur auf den lokalen Beobachtungen o_i trainiert. Nach Abschluss des Trainings wird der Critic verworfen und nur die dezentralen Policies ausgeführt. Das System bleibt damit zu Inferenzzeit vollständig dezentral einsetzbar [7].

Der entscheidende Vorteil bei dieser Konstellation liegt in der Stabilisierung des Lernprozesses. Da der Critic den globalen Zustand und die Aktionen aller Agenten kennt, ist die zugrundeliegende Markov-Annahme aus Sicht des Critic erfüllt, und die in Gleichung (23) dargestellte Nicht-Stationarität verschwindet auf der Trainingsebene. Die dezentralen Actors profitieren dabei indirekt von dieser Stabilität durch das konsistentere Gradientensignal des zentralen Critics. Über MADDPG hinaus wurde das Prinzip in zahlreichen weiteren Algorithmen aufgegriffen, darunter COMA [28], QMIX [27] und das im Folgenden behandelte MAPPO.

Parameter Sharing In MARL Systemen in denen die Agenten homogen sind, also in denen alle Agenten identische Aktions- und Beobachtungsräume besitzen, lässt sich der Trainingsaufwand durch *Parameter Sharing* drastisch reduzieren. Anstelle von N separaten Policy-Netzwerken $\pi_1 \dots \pi_N$ teilen sich alle Agenten dieselben Parameter θ und handeln damit nach einer gemeinsamen Policy π_θ . Während

der Ausführung wird jeder Agent durch identische Gewichte gesteuert, unterscheidet sich aber durch seine individuelle Beobachtung. Teils wird auch ein Identifikator für die Agenten als zusätzliches Eingangsmerkmal mitgeführt, um Symmetriebrüche zu erlauben [29].

Eingeführt wurde dieses Verfahren in der Deep-MARL Literatur durch Gupta, Egorov und Kochenderfer [30], die unter dem Namen *PS-TRPO* eine parameterteilende Variante von *TRPO* für homogene Schwarmaufgaben vorgeschlagen haben. Empirisch zeigt sich, dass Parameter Sharing gegenüber unabhängig trainierten Policies sowohl die Stichprobeneffizienz als auch die finale Performance signifikant verbessert [21], [29]. Aus theoretischer Sicht entspricht das Parameter Sharing einer starken Strukturannahme über die Policies. Es wird die A-Priori Annahme der Austauschbarkeit homogener Agenten direkt in der Architektur eingebunden und reduziert dadurch den effektiven Hypothesenraum von N voneinander unabhängigen Policies auf eine einzige Policy. Genau diese Annahme ist eine zentrale Voraussetzung für die Anwendung Mean-Embedding basierter Repräsentation, welche in Abschnitt 2.5 behandelt werden.

Multi-Agent Proximal Policy Optimization MAPPO [21] ist die in der vorliegenden Arbeit zur Anwendung kommende Multi-Agent variante von PPO. Sie kombiniert die oben genannten Prinzipien des zentralen Critics im Sinne von CTDE und vollständiges Parameter Sharing zwischen homogenen Agenten. Konkret wird eine geteilte Policy $\pi_\theta(a_i | o_i)$ und eine zentralisierte Value-Function $V_\phi(s)$ trainiert, wobei s entweder den globalen Zustand oder eine Konkatenation aus globalem und lokalem Zustand bezeichnet. Die Policy wird durch das in Gleichung (20) eingeführte Clipped-Surrogate-Objective optimiert, der Advantage-Estimator $\hat{A}_t$ wird über die zentrale Value Function mittels GAE (Gleichung (16)) berechnet.

Die wichtigste empirische Erkenntnis von Yu, Velu, Vinitsky u. a. [21] in diesem Zusammenhang ist, dass PPO mit minimalem Hyperparameter-Tuning State-of-the-Art Performance auf den prominenten Benchmarks Multi-Agent Particle Environments, StarCraft Multi-Agent Challenge, Google Research Foobal und der Hanabi Challenge erreicht. Identifiziert werden fünf praxisrelevante Implementierungsfaktoren, die maßgeblich zur Performance beitragen. Value Normalisierung, agentenspezifische, zentrale Zustandseingaben für den Critic, Value-Clipping, eine begrenzte Anzahl an PPO-Update-Epochen sowie Death Masking in Umgebungen mit ausfallenden Agenten [21].

Für die in dieser Arbeit untersuchten Schwarmssysteme ist MAPPO aus drei Gründen die methodisch geeignete Wahl:

1. Das in Abschnitt 2.2 eingeführte PPO wird unverändert fortgeführt, sodass die Mean-Embedding-Architektur als Feature-Extraktor ohne algorithmische Modifikation in eine moderne, gut validierte Pipeline integriert werden kann.
2. Parameter Sharing ist bei MAPPO der Default, was sowohl der Homogenitätsannahme von Schwarmsystemen entspricht, als auch die Stichprobeneffizienz erhöht.
3. Die Verwendung eines modernen On-Policy Verfahrens stellt den Anschluss an den aktuellen Stand der Forschung sicher und vermeidet die in MADDPG bekannten Skalierungsgrenzen für große Agentenzahlen [5].

Da der Mean-Embedding Ansatz von Hüttenrauch, Šošić und Neumann [6] selbst als algorithmusagnostisch beschrieben wird, kann er dort verwendete Trägeralgorithmus TRPO ohne methodischen Bruch durch MAPPO ersetzt werden. Die ausführliche Begründung dieser Entscheidung erfolgt in Abschnitt 5

2.3.4 Mean Fields in MARL

Selbst unter der Anwendung von CTDE und Parameter Sharing stoßen klassische MARL-Algorithmen bei sehr großen Schwarmgrößen weiterhin an Grenzen bei der Skalierung. Der zentrale Critic in MADDPG und MAPPO erhält weiterhin den vollständigen Joining-Action-Vector $a_1, \dots, a_N$ als Eingabe, dessen Dimensionalität linear mit N wächst. Bei mehreren Dutzenden bis hunderten Agenten führt dies zu einem problematisch hochdimensionalen Eingaberaum und entsprechend hoher Stichprobenkomplexität [22].

Einen prinzipiell anderen Skalierungsansatz verfolgen *Mean Field MARL* Verfahren, die von Yang, Luo, Li u. a. [22] eingeführt wurden. Die Grundidee bei dieser Methode entspringt der Physik und

besteht darin, die paarweisen Interaktionen zwischen einem Agenten und allen anderen durch die Interaktion mit einer einzelnen *mittleren Aktion* der Nachbarschaft zu approximieren. Konkret wird die exakte Aktionswertfunktion

$$Q_i(s, a_1, \dots, a_N) \quad (25)$$

durch die Mean-Field-Approximation

$$Q_i(s, a_i, \bar{a}_i) \quad \text{mit} \quad \bar{a}_i = \frac{1}{|N(i)|} \sum_{j \in N(i)} a_j \quad (26)$$

ersetzt, in der $\bar{a}_i$ die mittlere Aktion der Nachbarn von Agent i in dessen Nachbarschaft $N(i)$ bezeichnet. Aus dem exponentiellen joint-Action-Space wird auf diese Weise ein Eingaberaum konstanter Dimension, der unabhängig von der Agentenzahl ist. Yang, Luo, Li u. a. [22] entwickeln auf dieser Grundlage zwei konkrete Algorithmen, Mean-Field Q-Learning (**MF-Q**) und Mean-Field Actor-Critic (**MFAC**), und zeigen empirisch deren Skalierbarkeit auf Aufgaben mit bis zu mehreren hundert Agenten.

Konzeptionell weist Mean-Field-MARL eine starke strukturelle Verwandtschaft mit dem Mean-Embedding Ansatz auf, der im Zentrum dieser Arbeit steht. Beide Verfahren ersetzen eine explizite Modellierung aller Agenteninteraktionen durch eine *aggregierte, kardinalitätsunabhängige Statistik*, die als kompakte Eingabe in eine nachgelagerte Policy oder Value-Function einfließt. Der entscheidende Unterschied liegt in der Wahl der aggregierten Größe und im Aggregationsmechanismus.

Mean-Field-MARL mittelt die *Aktionen* der Nachbarn im ursprünglichen Aktionsraum, während Mean-Embedding Verfahren die *Beobachtungen* der Nachbarn zunächst über ein gelerntes ϕ -Netzwerk in einen Merkmalsraum projizieren und dort dann mitteln. Mean-Embeddings können in diesem Sinne als Verallgemeinerung der Mean-Field-Idee auf einen gelernten, aufgabenadaptiven Merkmalsraum verstanden werden.

Mit dieser Verwandtschaft bildet Mean-Field-MARL einerseits eine wichtige konzeptionelle Brücke zwischen klassischen MARL Methoden und der in den folgenden Abschnitten behandelten Schwarmsystem-Repräsentation, andererseits aber auch eine alternative Skalierungsstrategie, welche in der Diskussion verwandter Arbeiten in Kapitel 3 ausführlicher eingeordnet wird. Im Folgenden werden zunächst Schwarmssysteme als Anwendungsgebiet eingeführt, bevor in Abschnitt 2.5 die mathematischen Grundlagen permutationsinvarianter Aggregation behandelt werden.

2.4 Schwarmssysteme

Schwarmssysteme bilden eine spezielle Klasse innerhalb des MARL Kosmos, in der typischerweise eine große Anzahl einfacher, homogener Agenten ohne eine zentrale Steuerung interagieren und durch lokale Wechselwirkungen ein kollektives Verhalten erzeugt um eine Aufgabe zu lösen [3], [4]. Die Forschungsrichtung der Schwarmrobotik entstand in den frühen 2000er Jahren als eigenständige Disziplin an der Schnittstelle zwischen verteilten Systemen, Robotik und kollektiver Intelligenz, stark inspiriert durch biologische Vorbilder wie Ameisenkolonien, Bienenschwärme oder Fischschulen [31].

Aus Sicht des in den vorhergehenden Abschnitten eingeführten MARL Formalismus stellen Schwarmssysteme eine besonders anspruchsvolle Anwendungsdomäne dar, da sie die kooperative Dec-POMDP Struktur mit einer expliziten Forderung nach *Skalierbarkeit auf sehr große Agentenzahlen*, der Notwendigkeit zur Verarbeitung *variabler Schwarmgrößen* sowie der *Austauschbarkeit homogener Agenten* kombiniert.

Diese drei Eigenschaften gehen über die in Abschnitt 2.3 behandelten allgemeinen Anforderungen an MARL hinaus und motivieren die in der vorliegenden Arbeit untersuchten strukturellen Repräsentationsannahmen.

Im Folgenden werden zunächst die definitorischen Eigenschaften von Schwarmsystemen nach Şahin [31] formal eingeführt (Abschnitt 2.4.1). Anschließend wird die etablierte Taxonomie kollektiver Verhaltensweisen nach Brambilla, Ferrante, Birattari u. a. [3] und Schranz, Umlauf, Sende u. a. [4] vorgestellt, die den Lösungsraum möglicher Schwarm-Aufgaben strukturiert (Abschnitt 2.4.2). Abschließend werden dann aus den definierten Eigenschaften die strukturellen Anforderungen an eine geeignete Zustandsrepräsentation abgeleitet (Abschnitt 2.4.3), welche unmittelbar zum mathematischen Fundament permutationsinvarianter Aggregation in Abschnitt 2.5 überleitet.

2.4.1 Definition und Eigenschaften

Die heute geläufige Definition der Schwarmrobotik geht auf Şahin [31] zurück:

"Swarm robotics is the study of how large numbers of relatively simple physically embodied agents can be designed such that a desired collective behavior emerges from the local interactions among agents and between the agents and the environment."

Diese Definition hebt für die vorliegende Arbeit drei zentrale Eigenschaften hervor: Die hohe Agentenzahl, die einfach gehaltenen Fähigkeiten eines einzelnen Agenten und das Konzept der Emergenz globalen Verhaltens aus rein lokalen Interaktionen.

Şahin [31] identifiziert weiter eine Reihe definierender Kriterien, mithilfe derer man Schwarmssysteme von verwandten „Normalen“ Systemen mit mehreren Agenten abgrenzen kann [31, S.12]:

- **Autonomie:** Die Agenten verfügen über eigene Sensorik und Aktorik, nehmen also eigenständig wahr und steuern sich selbst, und agieren ohne zentrale Kontrollinstanz.
- **Große Zahl:** Das System ist auf eine hohe Anzahl von Agenten ausgelegt, was bedeutet, dass die kollektive Lösung für viele Agenten skalieren soll.
- **Homogenität:** Der Schwarm besteht aus identischen oder wenigen homogenen Gruppen von Agenten. Es darf also keine zentrale Kontrollinstanz oder stark variierende Agenten geben.
- **Beschränkte Einzelfähigkeiten:** Im Verhältnis zur kollektiven Aufgabe sollen Sensorik, Kommunikations- und Rechenfähigkeiten stark begrenzt sein. Das zwingt die Agenten zur Kooperation.
- **Lokalität:** Sensorik und Kommunikation erfolgen ausschließlich lokal; globale Informationen stehen einzelnen Agenten nicht zur Verfügung.

Aus diesen Kriterien ergeben sich nach Şahin [31] drei *wünschenswerte* Eigenschaften, welche einen Schwarm *im Idealfall* auszeichnen und in der Literatur seither als zentrale Designziele gelten:

Robustheit bezeichnet die Fähigkeit des Systems, trotz des Ausfalls einzelner Agenten oder Störungen weiterhin funktionsfähig zu bleiben, da kein einzelner Agent kritisch für das Funktionieren des Gesamtsystems ist [3].

Flexibilität beschreibt die Anpassungsfähigkeit des Schwarms an veränderte Aufgaben oder Umweltbedingungen, ohne dass die einzelnen Agenten umprogrammiert werden müssen. Die kollektive Reaktion entsteht aus den lokalen Verhaltensregeln [4].

Skalierbarkeit bezieht sich auf die Eigenschaft, dass dieselben lokalen Verhaltensregeln über einen weiten Bereich von Schwarmgrößen funktionieren. Sowohl kleine Versuchsaufbauten mit wenigen Agenten als auch großflächige Anwendungen mit hunderten oder tausenden Agenten [3].

Insbesondere die Eigenschaften *Homogenität* und *Skalierbarkeit* sind für später in der Methodik der Arbeit relevant. Homogenität rechtfertigt die Verwendung von Parameter Sharing (Abschnitt 2.3.3) und impliziert die Austauschbarkeit einzelner Agenten in der Repräsentation. Skalierbarkeit erfordert eine Architektur, deren Komplexität nicht mit der Agentenzahl anwächst. Beide Eigenschaften werden in Abschnitt 2.4.3 formal als strukturelle Anforderungen an die Zustandsrepräsentation präzisiert.

2.4.2 Taxonomie kollektiver Verhaltensweisen

Über die definierten Eigenschaften hinaus existiert in der Literatur eine etablierte Taxonomie der kollektiven Verhaltensweisen, die ein Schwarmsystem realisieren kann. Die heutzutage in der Schwarmrobotik typische Klassifikation geht auf Brambilla, Ferrante, Birattari u. a. [3] zurück und wurde von Schranz, Umlauf, Sende u. a. [4] um aktuelle Forschungsgebiete erweitert. Beide Taxonomien teilen die Schwarm-Aufgaben in vier verschiedene Hauptkategorien ein: Räumliche Organisation, Navigation, kollektive Entscheidungsfindung und weitere, übergreifende Verhaltensweisen.

Räumliche Organisation Verhaltensweisen dieser Kategorie betreffen die räumliche Anordnung der Agenten zueinander [3]. Klassische Beispiele sind *Aggregation*, bei der sich der Schwarm an einem gemeinsamen Ort versammelt, *Pattern Formation*, bei der bestimmte Muster realisiert werden müssen, sowie *Self-Assembly*, bei der sich Agenten physisch zu größeren Strukturen verbinden.

Navigation Navigation umfasst alle Aufgaben, in denen sich der Schwarm koordiniert durch den Raum bewegen muss [3]. Hierzu zählen beispielsweise *Flocking* (Die gemeinsame gerichtete Bewegung wie bei Vogelschwärmen), *Foraging* (Suchen und Sammeln von Ressourcen), *Collective Exploration* (Gemeinsame Erkundung von unbekannten Gebieten) und *Coordinated Motion*, in dessen Rahmen auch Verfolgungsaufgaben wie das in MARL häufige *Pursuit-Evasion* Szenarios angesiedelt sind.

Kollektive Entscheidungsfindungen Diese Kategorie beschreibt Aufgaben, in denen der Schwarm eine gemeinsame Wahl trifft, ohne dass eine zentrale Instanz dies koordiniert [3]. Wesentliche Beispiele sind *Consensus Achievement*, das Erreichen einer einheitlichen Entscheidung unter allen Agenten, und *Task Allocation*, die verteilte Zuordnung von Teilaufgaben an einzelne Agenten.

Übergreifende und weitere Verhaltensweisen Schranz, Umlauf, Sende u. a. [4] erweitern die Taxonomie von Brambilla, Ferrante, Birattari u. a. [3] um eine Reihe in den letzten Jahren prominenter Forschungsthemen, darunter zum Beispiel *Collective Localization*, die gemeinsame Positionsbestimmung ohne externe Informationen, *Collective Perception*, die verteilte Erkennung von Umweltmerkmalen, *Synchronization*, die zeitliche Abstimmung periodischer Aktionen, *Self-Healing*, die Wiederherstellung der Funktionsfähigkeit nach Ausfällen von einzelnen Agenten, als auch *Human-Swarm Interaction* als Schnittstelle zwischen Mensch und Schwarm.

Die für die vorliegende Arbeit relevanten Aufgaben fallen in die Kategorien räumliche Organisation und Navigation. Die konkrete Auswahl und Begründung erfolgt in Kapitel 5. Unabhängig von der konkreten Aufgabe ergeben sich aus der gemeinsamen Strukturannahme homogener, lokal agierender und in variabler Zahl auftretender Agenten jedoch identische Anforderungen an die zugrundeliegende Zustandsrepräsentation. Diese Anforderungen werden im folgenden Abschnitt formal herausgearbeitet.

2.4.3 Strukturelle Anforderungen an die Repräsentation

Aus den Eigenschaften aus Abschnitt 2.4.1, die einen Schwarm definieren, lassen sich unabhängig von der konkreten kollektiven Aufgabe drei strukturelle Anforderungen an die Zustandsrepräsentation einzelner Agenten ableiten. Diese Anforderungen bilden quasi eine Checkliste, welche eine geeignete Beobachtungsaggregation für MARL in Schwarmsystemen erfüllen muss.

Permutationsinvarianz Homogenität als Eigenschaft impliziert, dass alle Agenten austauschbar sind. Eine Umnummerierung der Agenten ändert weder die physikalische Situation noch das optimale Verhalten eines beobachtenden Agenten. Eine geeignete Repräsentationsfunktion f muss diese "Symmetrie" respektieren. Für jede beliebige Permutation π der Eingabemenge $X = \{x_1, \dots, x_N\}$ muss

$$f(X) = f(\pi(X)) \quad (27)$$

gelten [32]. Ein einfacher Konkatenationsansatz, bei dem die Beobachtungen anderer Agenten in einer festen Reihenfolge in den Eingabevektor geschrieben werden, verletzt also diese Eigenschaft. Das Netzwerk könnte ungewollt künstliche Abhängigkeiten der Anordnung lernen.

Kardinalitätsunabhängigkeit Die Skalierbarkeitseigenschaft fordert, dass dieselbe Repräsentation und Policy über einen weiten Bereich von Schwarmgrößen N einsetzbar ist. Dies setzt voraus, dass die Eingabedimension der Policy *nicht* mit N mitwächst. Anderenfalls müsste für jede Schwarmgröße ein eigenes Modell trainiert werden. Eine geeignete Repräsentationsfunktion f muss also eine *Aggregation* über die Menge der beobachteten Agenten realisieren, deren Ausgabe in einem festen Raum $\mathbb{R}^k$ liegt:

$$f : \bigcup_{N \in \mathbb{N}} \mathcal{X}^N \rightarrow \mathbb{R}^k. \quad (28)$$

Dieselbe Policy kann dann unverändert auf Schwärme beliebiger Größe angewendet werden. Diese Eigenschaft wird in dieser Arbeit durch einen Transfer von Trainings- auf Testgröße untersucht.

Lokale Beobachtbarkeit Aus der Lokalitätseigenschaft geht hervor, dass die für die Aggregation verfügbare Eingabemenge nicht den gesamten Schwarm, sondern nur eine Teilmenge $N(i) \subseteq \{1, \dots, N\}$ in Sensorreichweite des Agenten i umfasst. Diese Nachbarschaft kann sich über die Zeit ändern, sodass die Repräsentationsfunktion zusätzlich robust gegenüber Änderungen der Mengenzusammensetzung sein muss. Diese Eigenschaft ergibt sich direkt aus der Kardinalitätsunabhängigkeit, sofern die Nachbarschaft als zeitvariable, endliche Menge interpretiert wird.

Diese drei Anforderungen sind miteinander verwoben, aber prinzipiell unterschiedlich. Sie definieren gemeinsam eine Klasse zulässiger Aggregationsfunktionen für Schwarmbeobachtungen. Klassische Multi-Layer-Perceptron (MLP)s erfüllen keine dieser Eigenschaften, da sie eine feste Eingabedimension erwarten und sensitiv gegenüber der Reihenfolge ihrer Eingaben sind. Eine strukturell angemessene Lösung erfordert eine explizit auf Mengen operierende Funktionsklasse, deren mathematische Grundlagen und konkrete Realisierung Gegenstand des folgenden Abschnittes 2.5 sind.

2.5 Permutationsinvarianz und Kernel Mean Embeddings

Aus den im Abschnitt 2.4.3 formulierten strukturellen Anforderungen ergibt sich die zentrale mathematische Aufgabe dieses Kapitels. Die Aufgabe ist es, eine Klasse von Funktionen zu finden, die endliche Mengen variabler Kardinalität auf permutationsinvariante, festdimensionale Repräsentationen abbildet. Für diese Aufgabe existieren in der Literatur zwei zunächst unabhängig entwickelte Antworten, die sich in den letzten Jahren als zwei Seiten der selben Idee herauskristallisiert haben. Die klassische, auf reproduzierenden Kernen basierende Theorie der *Kernel Mean Embeddings* [33] und die neuronale Architektur der *Deep Sets* [32].

Klassische Kernel mean Embeddings bilden eine Wahrscheinlichkeitsverteilung auf ein Element eines Reproducing Kernel Hilbert Space (RKHS) ab und verwenden einen festen, mathematisch vorgegebenen Kern als Feature-Map. Deep Sets ersetzen diesen festen Kern durch ein lernbares neuronales Netzwerk und realisieren die Aggregation als elementweise Summe oder Mittelung über die gelernten Repräsentationen. Beide Ansätze erfüllen die in Abschnitt 2.4.3 definierten Anforderungen, unterscheiden sich jedoch in der Wahl des Merkmalsraums und in der theoretischen Charakterisierung ihrer Repräsentationskapazität.

Der folgende Aufbau dieses Abschnitts spiegelt diese zweisträngige Theorie wider. Nach einer formalen Einführung permutationsinvarianter Funktionen auf Mengen (Abschnitt 2.5.1) werden zunächst die Grundlagen von RKHS (Abschnitt 2.5.2) und darauf aufbauend die klassischen Kernel Mean Embeddings behandelt (Abschnitt 2.5.3). Anschließend führt Abschnitt 2.5.4 die neuronale Deep-Sets-Architektur ein, deren theoretische Schranken in Abschnitt 2.6 diskutiert werden. Abschnitt 2.6.1 skizziert über das Neural Tangent Kernel Resultat eine theoretische Brücke zwischen den beiden Ansätzen. Abschließend stellt Abschnitt 2.7 die konkrete Realisierung von Mean Embeddings für MARL in Schwarmsystemen nach Hüttenrauch, Šošić und Neumann [6] vor, die als Ausgangspunkt der empirischen Untersuchungen dieser Arbeit dient.

2.5.1 Permutationsinvariante Funktionen auf Mengen

Anders als klassische Neuronale Netze deren Eingabe als geordneter Vektor fester Dimension vorliegt, operiert die in dieser Arbeit untersuchte Repräsentation auf einer *Menge* mit einer variablen Größe. Dieser Unterschied ist nicht nur notationell, sondern fundamental. Ein Vektor $(x_1, \dots, x_N) \in X^N$ codiert die Reihenfolge der Elemente, wohingegen eine Menge $X = \{x_1, \dots, x_N\}$ die Reihenfolge nicht kodiert. Eine sinnvolle Repräsentation muss diesen strukturellen Unterschied beachten und einbinden.

Definitionen Ist $\mathcal{X}$ ein Eingaberaum (etwa $\mathbb{R}^d$ für d -dimensionale Beobachtungsvektoren), S_N die symmetrische Gruppe aller Permutationen von $\{1, \dots, N\}$ und $\pi \in S_N$ eine Permutation, die auf Tupel über $\mathcal{X}$ durch $\pi(x_1, \dots, x_N) = (x_{\pi(1)}, \dots, x_{\pi(N)})$ wirkt. Eine Funktion $f : \mathcal{X}^N \rightarrow \mathbb{R}^k$ heißt *permutationsinvariant*, wenn

$$f(x_1, \dots, x_N) = f(x_{\pi(1)}, \dots, x_{\pi(N)}) \quad \forall \pi \in S_N \quad (29)$$

gilt [32]. Die Funktion liefert für jede Umordnung derselben Eingabe denselben Output. Die Reihenfolge der Elemente ist also für das Resultat irrelevant. Eine permutationsinvariante Funktion ist äquivalent zu einer Funktion, die auf der zugehörigen Menge definiert ist.

In Abgrenzung dazu heißt eine Funktion $g : \mathcal{X}^N \rightarrow \mathcal{X}^N$ *permutationsäquivalent*, wenn

$$g(\pi(x_1, \dots, x_N)) = \pi(g(x_1, \dots, x_N)) \quad \forall \pi \in S_N \quad (30)$$

gilt [32]. Während eine permutationsinvariante Funktion eine Menge auf einen einzigen Ausgabewert abbildet, transformiert eine permutationsäquivalente Funktion eine Menge in eine andere Menge gleicher Kardinalität, wobei die Korrespondenz zwischen Eingabe- und Ausgabeelementen erhalten bleibt. Für die in dieser Arbeit untersuchte Repräsentation eines Schwarms durch einen einzelnen festdimensionalen Vektor ist die Permutationsinvarianz die relevante Eigenschaft. Die Äquivarianz wird beispielsweise in Graph Neural Networks (GNN)s als eine strukturelle Annahme relevant, welche in Kapitel 3 weiter diskutiert werden.

Variable Mengengröße Permutationsinvarianz allein genügt leider nicht, um die in Abschnitt 2.4.3 definierte Kardinalitätsunabhängigkeit zu erreichen. Erforderlich ist zusätzlich, dass die Repräsentationsfunktion auf Mengen *unterschiedlicher Größe* operieren kann. Formal bedeutet das, dass die zugrundeliegende Funktion auf der Vereinigung aller endlichen Tupel über $\mathcal{X}$ definiert sein muss

$$f : \bigcup_{N \in \mathbb{N}} \mathcal{X}^N \rightarrow \mathbb{R}^k, \quad \text{mit (29) für jedes feste } N. \quad (31)$$

Funktionsklassen, die sowohl die Permutationsinvarianz als auch die variable Eingabekardinalität bei konstanter Ausgabedimension erfüllen, werden in der Literatur unter dem Begriff *Set Functions* zusammengefasst [32], [34]. Die zentrale Frage ist nun, wie sich Set Functions in der Praxis konstruieren und parametrisieren lassen. Die folgenden Abschnitte stellen zwei wesentliche Antworten auf diese Frage vor. Zuerst die klassische, kernelbasierte Konstruktion über RKHSs und die neuronale Konstruktion über elementweise Feature-Maps mit anschließender Aggregation.

2.5.2 Reproducing Kernel Hilbert Spaces

Die klassische, kernelbasierte Konstruktion von permutationsinvarianten Mengenrepräsentationen basiert auf der Theorie der Reproducing Kernel Hilbert Space (RKHS). Diese Theorie geht auf Aronszajn [35] zurück und bildet seit den späten 1990er Jahren das mathematische Fundament eines Großteils der modernen Kernelmethoden im maschinellen Lernen [36], [37]. Im Folgenden werden die für diese Arbeit relevanten Konzepte eingeführt. Darunter fallen was reproduzierbare Kerne sind, die Feature-Maps, der Kernel-Trick sowie typische Kerne und das Maximum Mean Discrepancy als zugehöriges Distanzmaß.

Hilbert-Räume und reproduzierende Kerne Ein Hilbert-Raum $\mathcal{H}$ über $\mathbb{R}$ ist ein vollständiger Vektorraum mit einem Skalarprodukt $\langle \cdot, \cdot \rangle_{\mathcal{H}}$, das eine Norm $\|f\|_{\mathcal{H}} = \sqrt{\langle f, f \rangle_{\mathcal{H}}}$ induziert. Im Kontext von RKHS ist $\mathcal{H}$ ein Raum reellwertiger Funktionen $f : \mathcal{X} \rightarrow \mathbb{R}$ auf dem Eingaberaum $\mathcal{X}$. Ein RKHS zeichnet sich dadurch aus, dass die Punktauswertung $f \mapsto f(x)$ für jedes feste $x \in \mathcal{X}$ eine stetig lineare Funktion. Aus dem Satz von Riesz folgt damit die Existenz einer eindeutigen Funktion $k_x = k(x, \cdot) \in \mathcal{H}$, für die *reproduzierende Eigenschaft*

$$f(x) = \langle f, k(x, \cdot) \rangle_{\mathcal{H}} \quad \forall f \in \mathcal{H}, x \in \mathcal{X} \quad (32)$$

gilt [35]. Die zugehörige Funktion $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ heißt *reproduzierender Kern* und ist symmetrisch sowie positiv semidefinit. Umgekehrt korrespondiert nach dem Theorem von Moore-Aronszajn jeder symmetrische, positiv semidefinite Kern eindeutig mit einem RKHS, sodass die Wahl eines Kerns die Wahl eines zugehörigen Funktionenraums vollständig festlegt [35].

Feature-Map und Kernel-Trick Aus der reproduzierenden Eigenschaft (32) folgt unmittelbar die Existenz einer *Feature-Map*

$$\varphi : \mathcal{X} \rightarrow \mathcal{H}, \quad x \mapsto k(x, \cdot), \quad (33)$$

die jedes Eingabeelement auf eine Funktion im RKHS abbildet. Das Skalarprodukt zweier solcher Bilder reduziert sich aufgrund der reproduzierenden Eigenschaft auf eine einzige Kernel-Auswertung

$$\langle \varphi(x), \varphi(y) \rangle_{\mathcal{H}} = k(x, y). \quad (34)$$

Diese Identität ist als *Kernel-Trick* bekannt [36] und ermöglicht es, im potenziell unendlich-dimensionalen Raum $\mathcal{H}$ zu rechnen, ohne φ jemals explizit auszuwerten. In der Praxis wählt man einen Kern direkt, da sich der entsprechende Funktionsraum implizit ergibt. Typische Beispiele sind der lineare Kern $k(x, y) = x^\top y$ der polynomielle Kern $k(x, y) = (x^\top y + c)^d$ und der besonders weit verbreitete Gauß- oder Radial Basis Function (RBF)-Kern

$$k(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right), \quad (35)$$

dessen zugehöriger RKHS unendlich-dimensional ist und der unter zusätzlichen Bedingungen in der Klasse stetiger beschränkter Funktionen liegt [36]. Der RBF-Kern findet sich auch als eine der von Hüttenrauch, Šošić und Neumann [6] untersuchten Varianten der Mean-Embedding Repräsentation wieder.

Charakteristische Kerne und Maximum Mean Discrepancy Eine für die folgende Behandlung von Mean Embeddings zentrale Eigenschaft betrifft die Frage, wann zwei Wahrscheinlichkeitsverteilungen P und Q über $\mathcal{X}$ durch ihre Einbettung in den RKHS eindeutig unterschieden werden können. Ein Kern k wird *charakteristisch* genannt, wenn die zugehörige Abbildung $P \mapsto \mathbb{E}_{x \sim P}[k(x, \cdot)]$ injektiv ist, also unterschiedliche Verteilungen stets auf unterschiedliche Elemente des RKHS abgebildet werden [38]. Für charakteristische Kerne lässt sich auf der Menge der Wahrscheinlichkeitsverteilungen die Maximum Mean Discrepancy (MMD) als Distanz

$$\text{MMD}^2(P, Q) = \left\| \mathbb{E}_{x \sim P}[k(x, \cdot)] - \mathbb{E}_{y \sim Q}[k(y, \cdot)] \right\|_{\mathcal{H}}^2 \quad (36)$$

definieren [39]. Die MMD verschwindet genau dann, wenn $P = Q$ gilt und stellt damit ein vollwertiges Distanzmaß auf Verteilungen dar. Sriperumbudur, Gretton, Fukumizu u. a. [38] zeigen, dass jeder beschränkte, stetige und strikt positiv definite Kern charakteristisch ist.

Damit ist das mathematische Fundament gelegt, um im folgenden Abschnitt das Kernel Mean Embedding als zentrales Konzept der Arbeit einzuführen.

2.5.3 Kernel Mean Embeddings

Die im vorigen Abschnitt erwähnten Feature-Maps $\phi : \mathcal{X} \rightarrow \mathcal{H}$, $x \mapsto k(x, \cdot)$ ordnet jedem einzelnen Punkt $x \in \mathcal{X}$ ein Element des RKHS zu. *Kernel Mean Embeddings* verallgemeinern diese Form vom Punkt zur *Verteilung*. Jeder Wahrscheinlichkeitsverteilung P über $\mathcal{X}$ wird ein einzelnes Element μ_P in $\mathcal{H}$ zugeordnet, das die gesamte Verteilung in einer kompakten, funktionalen Form repräsentiert [33]. Intuitiv ist μ_P der „Schwerpunkt“ der eingebetteten Verteilung im RKHS, also das gewichtete Mittel der Feature-Map über alle Stichprobenpunkte, gewichtet mit der Wahrscheinlichkeitsdichte von P .

Diese Verallgemeinerung erlaubt es, die gesamte Maschinerie der Kernel-Methoden, die ursprünglich für Punkte konzipiert wurde, auf ganze Wahrscheinlichkeitsverteilungen anzuwenden. Innere Produkte, Distanzen und Projektionen im RKHS werden dadurch zu wohldefinierten Operationen auf Verteilungen. Insbesondere lassen sich endliche Stichproben aus einer Verteilung über das empirische Mean Embedding in genau den selben Raum einbetten und mit den theoretischen Größen vergleichen. Für die in dieser Arbeit untersuchte Anwendung auf Schwarmssysteme ist dies die entscheidende Eigenschaft, denn die Beobachtungen eines Agenten über seine Nachbarn kann man als Stichprobe aus einer Schwarmverteilung interpretieren.

Definition Definieren wir $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ als ein reproduzierender Kern mit zugehörigem RKHS $\mathcal{H}$ und P ein Wahrscheinlichkeitsmaß auf $\mathcal{X}$. Unter der Bedingung $\mathbb{E}_{x \sim P}[\sqrt{k(x, x)}] < \infty$ existiert das Kernel mean Embedding von P als Bochner-Integral [33]

$$\mu_P = \mathbb{E}_{x \sim P}[k(x, \cdot)] = \int_{\mathcal{X}} k(x, \cdot) dP(x) \in \mathcal{H}. \quad (37)$$

Das Bochner-Integral verallgemeinert das Lebesgue-Integral auf Banach-Raum-wertige Funktionen und ist hier notwendig, da der Integrand $x \mapsto k(x, \cdot)$ Werte im potenziell unendlich-dimensionalen Hilbert-Raum $\mathcal{H}$ annimmt. Die genannte Bedingung sichert die absolute Integrierbarkeit der Norm

und damit die Existenz des Integrals als Element von $\mathcal{H}$. Das resultierende μ_P ist eine reelwertige Funktion auf $\mathcal{X}$, deren Auswertung an einem Punkt $y \in \mathcal{X}$ den Erwartungswert der kernelfunktion liefert $\mu_P(y) = \mathbb{E}_{x \sim P}[k(x, y)]$.

Reproduzierende Eigenschaft und Erwartungswert-Identität Eine zentrale Erkenntnis der Definition (37) ist die für alle $f \in \mathcal{H}$ gültige Identität

$$\langle f, \mu_P \rangle_{\mathcal{H}} = \mathbb{E}_{x \sim P}[f(x)]. \quad (38)$$

Diese geht aus der reproduzierenden Eigenschaft $\langle f, k(x, \cdot) \rangle_{\mathcal{H}} = f(x)$ aus Abschnitt 2.5.2 durch Vertauschen von Erwartungswert und innerem Produkt hervor. Das ist unter der oben genannten Integrabilitätsbedingung zulässig

$$\langle f, \mu_P \rangle_{\mathcal{H}} = \left\langle f, \mathbb{E}_{x \sim P}[k(x, \cdot)] \right\rangle_{\mathcal{H}} = \mathbb{E}_{x \sim P}[\langle f, k(x, \cdot) \rangle_{\mathcal{H}}] = \mathbb{E}_{x \sim P}[f(x)]. \quad (39)$$

Die Gleichung (38) ist von zentraler Bedeutung. Sie zeigt, dass μ_P als ein *linearer Erwartungswert-Operator* auf $\mathcal{H}$ agiert. Jede Größe der Form $\mathbb{E}_{x \sim P}[f(x)]$ mit $f \in \mathcal{H}$ ist vollständig durch μ_P bestimmt. In diesem Sinne kodiert μ_P alle Momente und Statistiken von P , die sich durch Funktionen aus dem RKHS ausdrücken lassen.

Injektivität und Verteilungsidentifikation Damit das Embedding als vollwertige Repräsentation von Verteilungen dienen kann, muss die Abbildung $P \mapsto \mu_P$ injektiv sein. Das bedeutet, dass unterschiedliche Verteilungen immer auf unterschiedliche Elemente des RKHS abgebildet werden. Genau diese Eigenschaft definiert einen charakteristischen Kern, für welche die Verteilung P vollständig durch μ_P bestimmt ist [38]. Insbesondere der RBF-Kern, der unter den im vorigen Abschnitt genannten Bedingungen auch charakteristisch ist, garantiert damit, dass kein Informationsverlust durch die Einbettung entsteht. Zwei Verteilungen mit identischem Mean-Embedding sind dadurch immer identisch.

Die in Gleichung (36) eingeführte Maximum Mean Discrepancy (MMD)

$$\text{MMD}^2(P, Q) = \|\mu_P - \mu_Q\|_{\mathcal{H}}^2 \quad (40)$$

lässt sich dadurch als Distanz zwischen den Embeddings zweier Verteilungen im RKHS interpretieren. Für charakteristische Kerne ist MMD eine vollwertige Metrik auf dem Raum der Wahrscheinlichkeiten.

Empirisches Mean Embedding In der Praxis ist die zugrunde liegende Verteilung P unbekannt, wobei jedoch eine endliche Stichprobe $X = \{x_1, \dots, x_N\}$ mit $x_i \stackrel{\text{iid}}{\sim} P$ verfügbar ist. Das empirische Kernel Mean Embedding ersetzt den Erwartungswert in (37) durch das arithmetische Mittel über die Stichproben

$$\hat{\mu}_X = \frac{1}{N} \sum_{i=1}^N k(x_i, \cdot) \in \mathcal{H}. \quad (41)$$

was als Estimator verstanden werden kann. P wird durch die empirische Verteilung $\hat{P}_N = \frac{1}{N} \sum_i \delta_{x_i}$ ersetzt, und (37) auf $\hat{P}_N$ angewendet ergibt unmittelbar (41). Smola, Gretton, Song u. a. [33] zeigen in ihrem zweiten Theorem, dass $\hat{\mu}_X$ unter der Bedingung $\mathbb{E}_{x \sim P}[k(x, x)] < \infty$ ein konsistenter Estimator für μ_P ist und mit hoher Wahrscheinlichkeit

$$\|\hat{\mu}_X - \mu_P\|_{\mathcal{H}} = O_p(N^{-1/2}) \quad (42)$$

gegen μ_P konvergiert.

Die Konvergenz aus (42) ist von zentraler Bedeutung für diese Arbeit. Sie ist *unabhängig von der Dimension d* des Eingaberaums $\mathcal{X}$, was sie im Gegensatz zu anderen dichte-basierten Verfahren zur Schätzung von Verteilungen, deren Konvergenz mit wachsendem d exponentiell verschlechtert, immun gegen den *Curse of Dimensionality* werden lässt. Mean Embeddings umgehen diese Schwierigkeit, da sie nicht versuchen, die Dichtefunktion zu rekonstruieren, sondern direkt eine Statistik im RKHS schätzen, deren Komplexität durch die Eigenschaften der Einheitskugel $\{f \in \mathcal{H} : \|f\|_{\mathcal{H}} \leq 1\}$ kontrolliert wird [33]. Diese Unabhängigkeit von der Dimension d ist eine wesentliche Motivation für den Einsatz von Mean-Embeddings als Repräsentation von Schwarmzuständen.

Strukturelle Relevanz für Mengenrepräsentationen Die Gleichung (41) erfüllt demnach die in 2.4.3 formulierten Anforderungen an eine Schwarmrepräsentation. Die Summierung über die Stichprobenelemente ist permutationsinvariant im Sinne von (29), denn das arithmetische Mittel der Summierung ist gegenüber Umordnungen der Indizes invariant. Außerdem ist $\bar{\mu}_X$ für jede endliche Mengengröße $N \geq 1$ definiert und liefert immer ein Element desselben Funktionsraums $\mathcal{H}$, sodass die geforderte Kardinalitätsunabhängigkeit erfüllt ist. Das empirische Kernel Mean Embedding stellt damit eine mathematisch fundierte Konstruktion einer Set Function im Sinne von (31) dar, deren Eigenschaften nicht heuristisch gefordert, sondern aus der RKHS-Theorie abgeleitet sind.

Übergang zur neuronalen Realisierung Klassische Kernel Mean Embeddings (KME) liefern eine vollständig, theoretisch fundierte Anleitung für permutationsinvariante Mengenrepräsentationen die zudem noch dimensionsunabhängig garantiert Konvergieren. Ihre praktische Anwendbarkeit ist jedoch durch zwei Faktoren eingeschränkt. Erstens ist der Merkmalsraum durch die Wahl des Kerns k fest vorgegeben und kann nicht an die jeweilige, eventuell von einer ersten Aufgabe abweichende, Aufgabe angepasst werden. Zweitens skaliert der explizite Umgang mit dem RKHS schlecht in der Anzahl der Stichproben und in der Dimensionalität der eingebetteten Informationen, wie auch Hüttenrauch, Šošić und Neumann [6] in ihrer Diskussion klassischer Kernel-Verfahren erläutern.

Eine naheliegende verallgemeinerung ersetzt die feste, mathematisch vorgegebene Feature-Map $k(x, \cdot)$ durch eine *lernbare* neuronale Feature-Map $\phi_\theta : X \rightarrow \mathbb{R}^k$ mit endlicher Ausgabedimension k . Das empirische Mean Embedding aus (41) geht dabei in die Mittelwertaggregation

$$\bar{\phi}_\theta(X) = \frac{1}{N} \sum_{i=1}^N \phi_\theta(x_i) \in \mathbb{R}^k \quad (43)$$

über. Diese neuronale Variante verliert zwar die direkten theoretischen Garantien der klassischen KMEs, kann sich allerdings an verschiedenste Aufgaben anpassen und ist unter Umständen Algorithmisch effizienter. Die theoretische Charakterisierung durch die Deep-Sets-Architektur [32] sowie ihre konkrete Anwendung im Schwarmkontext nach Hüttenrauch, Šošić und Neumann [6] sind Gegenstand der folgenden Abschnitte.

2.5.4 Deep Sets als neuronale Mean Embeddings

Die klassische KME-Theorie aus 2.5.3 liefert eine theoretisch fundierte Konstruktion permutationsinvarianter Mengenrepräsentationen, ist aber an einen fest vorgegebenen Kern gebunden. Zaheer, Kottur, Ravanbakhsh u. a. [32] schlagen mit der *Deep Sets* Architektur eine neuronale Verallgemeinerung vor, bei der die Feature-Map durch ein lernbares Netzwerk ersetzt wird und die Aggregation durch eine Summe oder einen Mittelwert über die transformierten Elemente erfolgt. Damit wird der Merkmalsraum nicht mehr durch eine mathematische Wahl, sondern durch den Lernprozess an die jeweilige Aufgabe angepasst.

Sum-Decomposition Eine Funktion f auf endlichen Mengen $X \subseteq \mathcal{X}$ heißt nach Zaheer, Kottur, Ravanbakhsh u. a. [32] *sum-decomposable*, wenn Funktionen $\phi : \mathcal{X} \rightarrow \mathcal{Z}$ und $\rho : \mathcal{Z} \rightarrow \mathbb{R}^d$ existieren, sodass $\phi : \mathcal{X} \rightarrow \mathcal{Z}$ und $\rho : \mathcal{Z} \rightarrow \mathbb{R}^d$ existieren, sodass

$$f(X) = \rho \left(\sum_{x \in X} \phi(x) \right). \quad (44)$$

gilt. Der Zwischenraum $\mathcal{Z}$ wird als *latenter Raum* bezeichnet. Die Struktur in (44) ist permutationsinvariant, da die Summe gegenüber Umordnungen invariant ist. ϕ wirkt elementweise auf jedem Mengenelement und extrahiert eine Repräsentation im latenten Raum, die Summation aggregiert diese Repräsentation permutationsinvariant, und ρ verarbeitet die aggregierte Größe zu einer aufgabenspezifischen Ausgabe.

Hauptresultat im abzählbaren Fall Zaheer, Kottur, Ravanbakhsh u. a. [32] zeigen, dass die Struktur in (44) eine vollständige Charakterisierung permutationsinvarianter Mengenfunktionen über abzählbaren

Grundmengen liefert. Konkret beweisen sie in ihrem zweiten Theorem die folgende Aussage:

$$\mathcal{X} \text{ abzählbar} \implies f : 2^{\mathcal{X}} \rightarrow \mathbb{R} \text{ permutationsinvariant} \iff f \text{ sum-decomposable über } \mathbb{R}. \quad (45)$$

Der Beweis nutzt die Abzählbarkeit von $\mathcal{X}$ aus, um über eine Bijektion $c : \mathcal{X} \rightarrow \mathbb{N}$ jedem Mengenelement eine natürliche Zahl zuzuordnen und mittels der Wahl von $\phi(x) = 4^{-c(x)}$ jede endliche Menge auf eine eindeutige reelle Zahl in Base-4 Darstellung abzubilden. Die Abbildung $\Phi(X) = \sum_{x \in X} \phi(x)$ ist dadurch injektiv, und $\rho = f \circ \Phi^{-1}$ liefert die Zerlegung [32].

Überabzählbarer Fall Für überabzählbare Grundmengen, wie z.B. $\mathcal{X} = \mathbb{R}^d$, beweisen Zaheer, Kottur, Ravanbakhsh u. a. [32] in ihrem siebten Theorem eine schwächere Aussage. Für *feste* Mengengrößen M ist jede stetige permutationsinvariante Funktion $f : [0, 1]^M \rightarrow \mathbb{R}$ stetig sum-decomposable über $\mathbb{R}^{M+1}$. Die Konstruktion verwendet das Tupel der Potenzsummen $\Phi_q(x) = \sum_{i=1}^M x_i^q$ für $q = 0, \dots, M$ und nutzt Newton-Girard-Identitäten, um die Injektivität auf der geordneten Untermenge zu zeigen. Im Gegensatz zum abzählbaren Fall ist die Existenz einer Zerlegung damit nur für Mengen fester Kardinalität garantiert, nicht für beliebige, endliche Teilmengen einer überabzählbaren Grundmenge.

Neuronale Realisierung und Verbindung zum Mean Embedding Werden ϕ und ρ als neuronale Netze implementiert, $\phi_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^k$ und $\rho_\psi : \mathbb{R}^k \rightarrow \mathbb{R}^{d'}$, so entsteht aus (44) die Deep-Sets Architektur. In Anwendung mit variabler Mengengröße wird die Summe häufig durch einen Mittelwert ersetzt

$$\bar{\phi}_\theta(X) = \frac{1}{|X|} \sum_{x \in X} \phi_\theta(x), \quad f_{\theta, \psi}(X) = \rho_\psi(\bar{\phi}_\theta(X)). \quad (46)$$

Diese Modifikation erhält die Permutationsinvarianz und sichert zusätzlich, dass der aggregierte Wert nicht mit der Mengengröße skaliert. Außerdem ist das die Form, die später auch bei Hüttenrauch, Šošić und Neumann [6] im Schwarmkontext verwendet wird.

Der Vergleich mit dem empirischen Kernel Mean Embedding (41) zeigt die Struktur der beiden Methoden. Das empirische KME aggregiert eine feste Feature-Map $k(x, \cdot)$ in einen unendlich-dimensionalen RKHS, wohingegen die neuronale Deep-Sets (46) eine gelernte, neuronale Feature-Map ϕ_θ in einen endlich-dimensionalen euklidischen Raum $\mathbb{R}^k$ aggregiert. Beide Strukturen sind permutationsinvariant, kardinalitätsunabhängig und folgen demselben Schema aus elementweiser Transformation und gemittelter Aggregation. In diesem Sinne lassen sich Deep-Sets als endlich dimensionale, gelernte Approximation des empirischen Kernel Mean Embeddings interpretieren, was auch Hüttenrauch, Šošić und Neumann [6] in ihrer Arbeit vorlegen.

Während diese Identifikation zwar eine gewisse Eleganz besitzt, geht durch den Übergang von $\mathcal{H}$ zu $\mathbb{R}^k$ eine wesentliche Eigenschaft verloren. Im klassischen Fall ist die Injektivität des Embeddings durch die Wahl eines charakteristischen Kerns garantiert, und die Dimension des Repräsentationsraums spielt keine Rolle, da $\mathcal{H}$ sowieso unendlich-dimensional ist. Im neuronalen Fall ist k ein freier Hyperparameter, und die Frage, welche Dimension k ausreichend ist, um permutationsinvariante Funktionen einer gegebenen Komplexität auszudrücken, wird zur zentralen Frage der Architekturwahl. Diese Frage wird im folgenden Abschnitt behandelt.

2.6 Repräsentationsdimension und untere Schranken

Das in Abschnitt 2.5.4 dargestellte Theorem von Zaheer, Kottur, Ravanbakhsh u. a. [32] garantiert die Existenz einer sum-decomposablen Darstellung für stetige permutationsinvariante Funktionen über $[0, 1]^M$ mit latenter Dimension $M + 1$. Es liefert damit eine hinreichende Bedingung, aber keine Aussage darüber, ob diese Dimension tatsächlich notwendig ist oder ob niedrigere Dimensionen ausreichen könnten. Wagstaff, Fuchs, Engelcke u. a. [34] schließen diese Lücke und zeigen, dass die Dimension des latenten Raums sowohl nach oben als auch nach unten scharf an die maximale Mengengröße gekoppelt ist.

Notwendige Bedingung Das zentrale Resultat von Wagstaff, Fuchs, Engelcke u. a. [34] ist eine untere Schranke für die latente Dimension. Konkret beweisen sie [34, Theorem 4.1]

$$M > N \implies \exists f : \mathbb{R}^M \rightarrow \mathbb{R} \text{ stetig, permutationsinvariant, nicht stetig sum-decomposable über } \mathbb{R}^N. \quad (47)$$

Die Beweisidee basiert auf einem topologischen Argument: für jede stetige sum-decomposition wäre die Abbildung $\Phi_M^{\text{ord}} : \mathbb{R}_{\text{ord}}^M \rightarrow \mathbb{R}^N$ eingeschränkt auf die geordnete Untermenge $\mathbb{R}_{\text{ord}}^M = \{x : x_1 < \dots < x_M\}$, eine stetige Injektion. Da $\mathbb{R}_{\text{ord}}^M$ homöomorph zu $\mathbb{R}^M$ ist, würde dies eine stetige Injektion $\mathbb{R}^M \hookrightarrow \mathbb{R}^N$ liefern, die für $M > N$ aus dimensionstopologischen Gründen nicht existieren kann [34]. Die Maximum-Funktion $\max(X)$ wird als konkretes Gegenbeispiel angeführt. Sie ist stetig und permutationsinvariant, aber für $N < M$ nicht stetig sum-decomposable.

Hinreichende Bedingung Wagstaff, Fuchs, Engelcke u. a. [34] verschärfen in Theorem 4.3 und 4.4 zugleich das hinreichende Resultat von Zaheer et al. von $\mathbb{R}^{M+1}$ auf $\mathbb{R}^M$ und erweitern es auf eine variable Mengengröße $\leq M$

$$f : \mathbb{R}^{\leq M} \rightarrow \mathbb{R} \text{ stetig} \iff f \text{ permutationsinvariant und stetig sum-decomposable über } \mathbb{R}^M. \quad (48)$$

Zusammen zeigen (47) und (48), dass die latente Dimension $N \geq M$ genau die Schranke, an der universelle Repräsentation stetiger permutationsinvarianter Funktionen auf Mengen der Größe bis M möglich wird.

Konsequenzen für die Architekturwahl Das Resultat aus der Arbeit von Wagstaff hat praktische Konsequenzen für Deep-Sets basierte Architekturen. Soll die Architektur in der Lage sein, beliebige stetige permutationsinvariante Funktionen über Mengen mit bis zu M Elementen darzustellen, muss die Dimension des latenten Raums mindestens M betragen. Wird die Dimension kleiner gewählt, existieren stetige Zielfunktionen, die von keiner Wahl der Parameter θ, ψ in (46) exakt darstellbar sind, und unabhängig von der Tiefe oder Breite der Netze ϕ_θ und ρ_ψ .

Wagstaff, Fuchs, Engelcke u. a. [34] betonen jedoch in der Diskussion ihrer Resultate, dass die Schranke $N \geq M$ eine Worst-Case Aussage über die *universelle* Approximationsfähigkeit ist und keine „echte“ Untergrenze für die Lösbarkeit konkreter Aufgaben darstellt. Bestimmte Funktionen sind über $\mathbb{R}$ sum-decomposable, also auch für $N \ll M$ exakt darstellbar. In ihrem empirischen Toy-Experiment zur Median-Schätzung beobachten die Autoren einen „kritischen Punkt“ der latenten Dimension, ab dem die Modellgüte saturiert. Dieser kritische Punkt liegt typischerweise *unterhalb* von M und skaliert ungefähr linear mit M [34]. Erklärt wird dies damit, dass reale Aufgaben Redundanz in den Eingabedaten enthalten und nicht die volle Komplexität beliebiger stetiger Funktionen verlangen.

Diese Beobachtung ist für die vorliegende Arbeit von zentraler Bedeutung, da sie nicht nur daraufhin deutet, dass die in Hüttenrauchs ursprünglichem Setup gewählte Embedding-Dimension von $k = 64$ vermutlich nicht durch die theoretische Worst-Case-Schranke motiviert ist, sondern empirisch festgelegt wurde und potenziell deutlich unter k liegende Werte für konkrete Schwarzaufgaben ausreichend sein könnten. Die systematische Untersuchung dieses Spielraums zwischen theoretischer Schranke und tatsächlich notwendiger Dimension ist Gegenstand der vorliegenden Arbeit.

2.6.1 Verbindung zu klassischen Kernen über den Neural Tangent Kernel

Die Parallele zwischen klassischen Kernel Mean Embeddings und neuronalen Deep-Sets ist nicht nur rein formal. Sie wird durch eine theoretische Linie gestützt, die in den letzten Jahren unter dem Stichwort Neural Tangent Kernel (NTK) erhebliche Aufmerksamkeit erfahren hat. Da diese Theorie für die vorliegende Arbeit nicht zentral ist und lediglich die konzeptionelle Nähe neuronaler und klassischer Mengenaggregation untermauert, wird sie hier nur kurz skizziert.

Jacot, Gabriel und Hongler [40] zeigen, dass das Training eines neuronalen netzes mit Gradientenabstieg im Grenzwert unendlicher Breite einem Kernel-Gradient-Descent entspricht. Der zugehörige Kern, der namensgebende Neural Tangent kernel, ist bei unendlicher Breite zufällig und während des Trainings veränderlich. Im „Infinite-Width-Limit“ konvergiert er gegen einen deterministischen Grenzkern und bleibt während des Trainings konstant [40]. ein hinreichend breites neuronales Netz verhält sich in diesem Umfeld also wie eine klassische Kernel-Methode mit dem NTK als zugrunde liegendem Kern.

Übertragen auf die in Abschnitt 2.5.4 eingeführte neuronale Feature-Map ϕ_θ ergibt sich daraus eine Brücke. Realisiert man ϕ_θ als hinreichend breites Netzwerk im NTK-Regime, so wird die gemittelte Aggregation (46) asymptotisch zu einem empirischen Mean Embedding bezüglich des NTK. Die in 2.5.3 entwickelte Theorie der Kernel Mean Embeddings überträgt sich damit unter geeignete Bedingungen auf die neuronale Realisierung.

Diese Ähnlichkeit hat allerdings zwei Einschränkungen, die für die vorliegende Arbeit interessant sind. Erstens gilt das Resultat von Jacot, Gabriel und Hongler [40] nur für eine spezielle Parametrisierungs- und Initialisierungsskalierung, was bedeutet, dass praktisch trainierte Netze davon abweichen. Zweitens findet im **NTK**-Limit asymptotisch kein Feature-Learning statt. Die internen Repräsentationen bleiben im Wesentlichen bei ihrer zufälligen Initialisierung. Genau das Gegenteil ist die Funktion von ϕ_θ in Hüttenrauchs Setup. Die Feature-Map soll aufgabenspezifische, durch das Lernsignal geformte Merkmale extrahieren, welche ein fester Kern nicht liefern kann.

Der **NTK** belegt damit, dass neuronale Mengenaggregation und klassische Kernel-Methoden nicht zwei völlig separate Gebiete sind, sondern Grenzfälle eines gemeinsamen Rahmens. Die in der vorliegenden Arbeit untersuchte empirische Frage, welche Embeddingdimensionen für konkrete Schwarzaufgaben ausreichend sind, ersetzt er nicht, da die untersuchten Netze gerade im Feature-Learning-Regime operieren, das der **NTK**-Limes nicht erfasst.

2.7 Mean Embeddings nach Hüttenrauch

Die auf in den vorigen Abschnitten eingegangene Theorie bildet die mathematische Grundlage für die konkrete Architektur, deren Skalierungsverhalten in dieser Arbeit untersucht wird. Hüttenrauch, Šošić und Neumann [6] schlagen eine spezifische Realisierung des neuronalen Mean Embeddings vor, die direkt an die klassische **KME**-Theorie anknüpft und gleichzeitig die strukturellen Anforderungen aus 2.4.3 erfüllt.

Wenn $O^i = \{o^{i,1}, \dots, o^{i,M_i}\}$ die Menge der Beobachtungen ist, die Agent i über seine M_i wahrgenommenen Nachbarn besitzt, dann kodiert jedes $o^{i,j} \in \mathbb{R}^d$ den Zustandsausschnitt eines beobachteten Agenten. Die aggregierte Repräsentation ist das empirische neuronale Mean Embedding

$$\phi^{\text{NN}}(O^i) = \frac{1}{|O^i|} \sum_{o^{i,j} \in O^i} \phi_\theta(o^{i,j}) \in \mathbb{R}^k, \quad (49)$$

wobei $\phi_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^k$ eine lernbare, durch ein neuronales Netz realisierte Feature-Map ist. In der konkreten Implementierung verwenden die Autoren ein einlagiges Feed-Forward Netzwerk mit $k = 64$ Neuronen und ReLU-Nichtlinearität,

$$\phi_\theta(o) = h(Wo + b), \quad W \in \mathbb{R}^{k \times d}, \quad b \in \mathbb{R}^k, \quad (50)$$

mit h als komponentenweiser ReLU [6]. $\phi^{\text{NN}}(O^i)$ dient als Eingabe für die Policy- und Value-Netzwerke des verwendeten TRPO-Algorithmus.

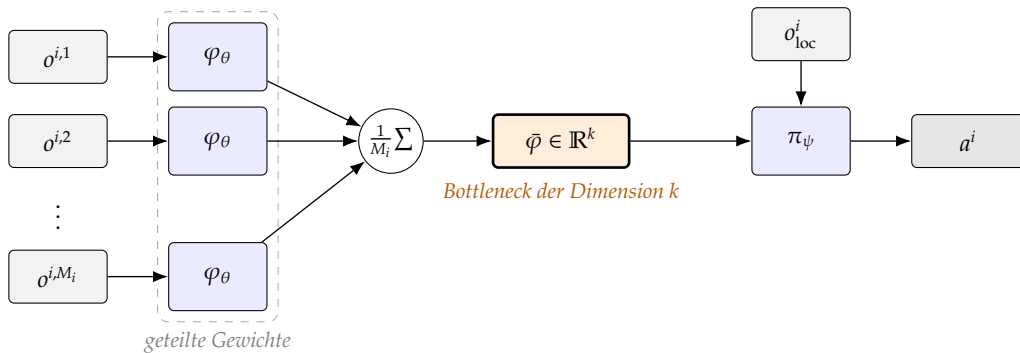


Abbildung 1: Mean-Embedding-Architektur nach Hüttenrauch, Šošić und Neumann [6]. Jede Nachbarbeobachtung $o^{i,j}$ wird durch dieselbe Feature-Map φ_θ verarbeitet (geteilte Gewichte), über die Menge gemittelt und ergibt den k -dimensionalen Mean-Embedding-Vektor $\bar{\varphi}$, der zusammen mit der eigenen Beobachtung o_{loc}^i in die Policy π_ψ eingeht. Die Dimension k bildet einen deterministischen Bottleneck zwischen der Nachbarschaftsmenge und der Policy-Ausgabe.

Die strukturellen Anforderungen aus 2.4.3 sind erfüllt, da die Permutationsinvarianz aus der Invarianz des arithmetischen Mittels entspringt, die Kardinalitätsunabhängigkeit aus der Normierung mit $|O^i|^{-1}$ hervortritt und sich die Robustheit gegenüber wechselnden Nachbarschaften daraus ergibt,

dass (49) für jede beliebige endliche Eingabemenge wohldefiniert ist und immer ein Element desselben Raums $\mathbb{R}^k$ liefert.

Strukturell entspricht (49) der Deep-Sets Architektur aus 2.5.4m allerdings mit einer entscheidenden Modifikation. Die Summenaggregation $\sum_{x \in X} \phi(x)$ wird durch die Mittelwertaggregation $|X|^{-1} \sum_{x \in X} \phi(x)$ ersetzt. Hüttenrauch, Šošić und Neumann [6] begründen diese Wahl explizit mit der Invarianz gegenüber der Agentenzahl. Im Gegensatz zur Summe, deren Norm linear mit der Mengengröße anwächst, bleibt der Wertebereich der gemittelten Aggregation unabhängig von $|O^i|$ - genau dadurch kann eine auf einer festen Schwarmgröße trainierte Policy auf größere oder kleinere Schwärme generalisieren, ohne dass das nachgelagerte Netzwerk Aggregatopmswerte Außerhalb seines Trainingsbereichs verarbeiten muss.

Die Mittelwertaggregation lässt sich nicht nur algorithmisch, sondern auch theoretisch deuten. Interpretiert man O^i als Stichprobe aus einer Verteilung P^i über Nachbarzustände ist (49) der gelernte Estimator für das Mean Embedding μ_{P^i} unter der gelernten Feature-Map ϕ_θ . Hüttenrauch, Šošić und Neumann [6] formulieren genau diese Interpretation und verweisen auf Smola, Gretton, Song u. a. [33] als theoretische Grundlage. Die in (42) festgehaltene Dimensionsunabhängigkeit der Konvergenzrate überträgt sich allerdings nicht unmittelbar, da ϕ_θ keine feste Feature-Map eines fixierten RKHS ist, sondern selbst von den Daten abhängt. Die NTK-Brücke aus 2.6.1 liefert hierzu eine Verbindung, aber keinen vollständigen Ersatz.

Damit eröffnet sich der zentrale Untersuchungsraum dieser Arbeit. Die Wahl $k = 64$ in der Originalarbeit ist nicht theoretisch motiviert, sondern empirisch festgelegt. In einer Ablationsstudie der Autoren berichten sie, dass die Embedding-Dimension einen sichtbaren, aber nicht dominanten Effekt auf die Lernkurve hat [6]. Die Worst-Case-Schranke $N \geq M$ aus [34] liefert eine theoretisch motivierte Untergrenze, ist aber durch die in [34] selbst beobachteten kritischen Punkte unterhalb von M in ihrer Aussagekraft für konkrete Aufgaben begrenzt. Wo sich die Lern- und Generalisierungsleistung der Hüttenrauch-Architektur verhält, wenn k systematisch unterhalb des Originalwerts variiert wird, und welche Dimension für konkrete Schwarmaufgaben tatsächlich erforderlich ist, sowie den Einfluss der Architektur des ϕ -Netzwerkes sind die zentralen empirischen Fragen, denen die folgenden Kapitel nachgehen.

3 Verwandte Arbeiten

2 hat die theoretischen Grundlagen permutationsinvarianter Mengenrepräsentationen, ihre Schranken nach Wagstaff, Fuchs, Engelcke u. a. [34] und die konkrete Mean-Embedding Architektur nach Hüttenrauch, Šošić und Neumann [6] eingeführt. Vor diesem Hintergrund verfolgt das vorliegende Kapitel nun zwei Ziele.

Erstens ordnet 3.1 die untersuchte Architektur in das Spektrum konkurrierender skalierungsinvarianter MARL-Ansätze ein.

Zweitens motiviert 3.2 die zentrale Forschungsfrage der Arbeit, die Wahl der Embedding Dimension k , durch Bezug zur Information Bottleneck Theorie und zu empirischen Befunden über Repräsentationskapazität in tiefen Netzen.

3.3 fasst abschließend die identifizierte Forschungslücke zusammen, in die sich diese Arbeit einordnet.

3.1 Skalierungsinvariante Architekturen in MARL

Die in 2.4.3 erwähnten Anforderungen (Permutationsinvarianz, Kardinalitätsunabhängigkeit und Robustheit gegenüber wechselnden Nachbarschaften) werden in der Literatur durch verschiedene Architekturklassen gelöst. Mean Embeddings stellen dabei nur eine von mehreren möglichen Designansätzen dar. Die folgenden Abschnitte stellen die im Schwarm- und MARL-Kontext relevanten Alternativen vor und zeigen, wie sich diese von der Mean-Embedding Architektur unterscheiden.

3.1.1 Permutationsinvariante Critics

Liu, Yeh und Schwing [41] adressieren in Permutation Invariant Critic (PIC) eine direkte Folge der in 2.5.1 diskutierten Symmetrieproblematik. Der zentrale Critic in MADDPG-artigen CTDE-Verfahren erhält Beobachtungen und Aktionen aller Agenten als konkatenierten Eingabevektor und ist damit sensitiv

gegenüber Permutationen, obwohl die zugrundeliegende Umgebung permutations-symmetrisch ist. [PIC](#) ersetzt den konkatenierenden [MLP-Critic](#) durch eine permutationsinvariante Aggregation auf Basis von Graph-Convolutional Networks mit Max-Pooling. Empirisch berichten die Autoren auf den Multi-Agent Paricle Environments eine Skalierung auf bis zu 30-fach größere Agenzanzahlen, sowie eine Verbesserung des Test>Returns um 15 - 50 % gegenüber [MADDPG](#) [41].

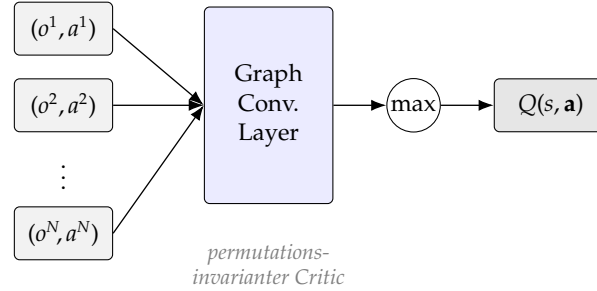


Abbildung 2: Architektur des Permutation Invariant Critic (PIC) nach Liu, Yeh und Schwing [41]. Beobachtungen und Aktionen aller Agenten werden durch eine Graph-Convolutional-Schicht mit anschließendem Max-Pooling zu einem permutationsinvarianten Q-Wert aggregiert. Die dezentralen Actors bleiben unverändert und operieren auf konkatenierten Eingaben.

[PIC](#) und der Mean-Embedding Ansatz argumentieren beide, dass das Erzwingen der Permutationssymmetrie als Architektur-Bias die effektive Hypothesenklasse reduziert und damit die Stichprobeneffizienz erhöht. Sie unterscheiden sich jedoch darin, wo genau die Invarianz greift. [PIC](#) macht den *Critic* invariant, lässt die dezentralen Actors aber auf konkatenierten Beobachtungen operieren. Hüttenrauch, Šošić und Neumann [6] wenden die Invarianz auf den *Observations der Actors* an, was die Voraussetzung dafür ist, dass dieselbe Policy auf wechselnde Schwarmgrößen angewandt werden kann.

3.1.2 Graph-basierte Ansätze

Eine zweite Technik modelliert das Multi-Agent System als Graph-Netzwerk $G = (V, E)$ mit Agenten als Knoten und die Interaktionen zwischen den Agenten als Kanten, und nutzt [GNNs](#) [42] zur Aggregation über die Nachbarschaft eines Knotens. Die Repräsentation eines Knotens i in schicht $\ell + 1$ wird als

$$h_i^{(\ell+1)} = \phi \left(h_i^{(\ell)}, \bigoplus_{j \in \mathcal{N}(i)} \psi(h_i^{(\ell)}, h_j^{(\ell)}) \right) \quad (51)$$

geupdated, wobei ψ eine Nachrichten- und ϕ eine Update-Funktion bezeichnen [42]. Die Aggregation $\bigoplus_{j \in \mathcal{N}(i)}$ erfolgt durch eine symmetrische Operation wie die Summe oder den Mittelwert. [GNNs](#) sind per Definition *permutationsäquivalent* $\text{GNN}(\pi(G)) = \pi(\text{GNN}(G))$. Knotenweises Auslesen kann sie jedoch auch permutationsinvariant machen.

Im [MARL](#)-Kontext wurden [GNNs](#) bereits in mehreren Architekturen aufgegriffen. Jiang, Dun, Huang u. a. [43] stellen mit Deep Graph Network ([DGN](#)) eine Architektur vor, in der iterierte Graph-Convolutional Layer mit Multi-Head-Attention als Faltungskerne eingesetzt werden, um Kooperationsmuster zwischen benachbarten Agenten zu lernen. Empirisch zeigt [DGN](#) auf Battle-Game- und Routing-Aufgaben Vorteile gegenüber [MADDPG](#), [MF-Q](#) und unabhängig trainierten Deep Q-Network ([DQN](#)) Agenten.

Vergleicht man den [GNN](#)-Ansatz mit Mean-Embeddings, so erkennt man, dass [GNNs](#) den Fokus von einer einmaligen, kompakten Aggregation hin zu einem mehrstufigen, lokalen Message-Passing. Das erlaubt eine explizite Modellierung von Interaktionsstrukturen, geht allerdings mit höherer architektonischer und rechnerischer Komplexität einher. In Betrachtung der zentralen Frage dieser Arbeit, wie viel Repräsentationsdimension pro Aggregationsschritt nötig ist, liefern auch [GNN](#) Arbeiten keine systematische Antwort, da in den oben genannten Antworten die latente Dimension heuristisch gewählt wird.

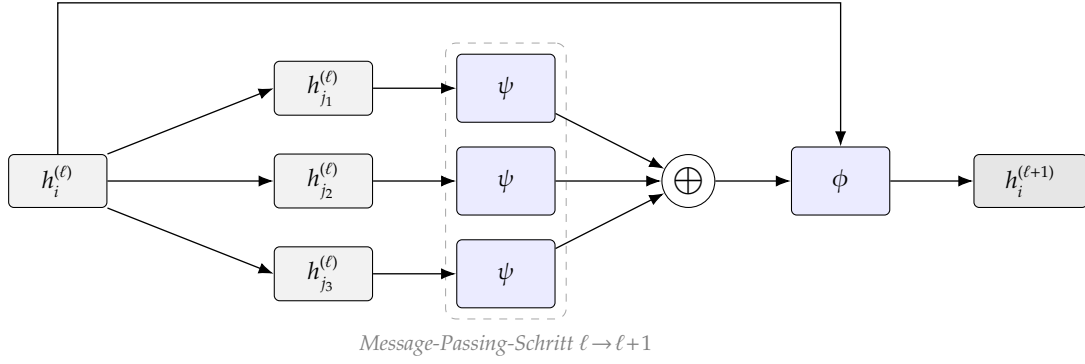


Abbildung 3: Funktionsprinzip eines Message-Passing-Schritts in einem Graph Neural Network nach Gilmer et al. [42]. Die Nachrichten-Funktion ψ (geteilte Gewichte) verarbeitet die Nachbarrepräsentationen $h_j^{(\ell)}$; eine symmetrische Aggregation $\oplus$ führt sie zusammen, und die Update-Funktion ϕ erzeugt zusammen mit $h_i^{(\ell)}$ die nächste Knotenrepräsentation $h_i^{(\ell+1)}$. DGN [43] setzt iterierte Schichten dieses Typs ein.

3.1.3 Attention basierte Aggregation

Attention basierte Modelle verallgemeinern die Mittelwertaggregation der Mean-Embeddings indem Eingabeelemente kontextabhängig gewichtet werden. Die Aggregation nimmt die allgemeine Form

$$f(X) = \sum_{x \in X} \alpha(x) \varphi(x), \quad \alpha(x_i) = \frac{\exp(e(x_i))}{\sum_{x_j \in X} \exp(e(x_j))} \quad (52)$$

an [44]. Bei elementweiser Definition der Gewichtungsfunktion $e(\cdot)$ und ohne Positionskodierung bleibt die Aggregation permutationsinvariant. Lee, Lee, Kim u. a. [45] zeigen mit dem *Set Transformer*, dass Self-Attention-basierte Architekturen eine universelle Klasse permutationsinvarianter Funktionen auf Mengen bilden.

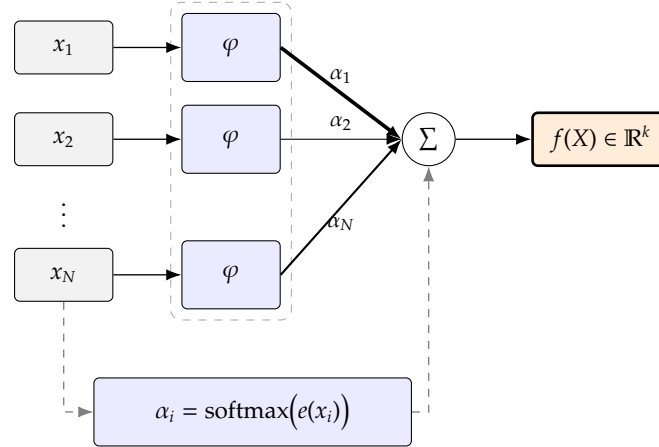


Abbildung 4: Attention-basierte permutationsinvariante Aggregation nach Vaswani et al. [44], wie sie im Set Transformer [45] und in MAAC [46] eingesetzt wird. Die Gewichte α_i werden datenabhängig aus den Eingabeelementen berechnet (gestrichelte Pfade) und gewichten die elementweisen Embeddings $\varphi(x_i)$ in der Summe. Im Gegensatz zur starren Mittelung der Mean-Embeddings (Abb. 1) ist die Aggregation adaptiv, kostet aber $O(N^2)$ Operationen für die paarweise Berechnung der Scores.

Im MARL-Kontext [46] in Multi-Actor-Attention-Critic (MAAC) Attention im zentralen Critic ein, um pro Agent die Beiträge der übrigen Agenten zu gewichten. Sie berichten eine konstantere Performance bei steigender Agentenzahl im Rover-Tower Benchmark gegenüber MADDPG und COMA Baselines.

Attention erreicht die zusätzliche Ausdruckskraft, indem die Komplexität verschoben wird. Statt einer einzigen Aggregation $O(|X|)$ werden paarweise Relevanzwerte berechnet, was im Self-Attention Fall $O(|X|^2)$ Operationen kostet. Für große Schwärme ist dies ein praktisch relevanter Nachteil. Aus Sicht der vorliegenden Arbeit ist Attention zwar nicht die sparsamere, sondern die ausdrucksstärkere Alternative zu Mean-Embeddings. Untersuchungen, wie weit sich die einfachere Mittelwert-aggregation reduzieren lässt, sind damit logischerweise der erste Schritt vor einem Einsatz teurerer Methoden.

3.1.4 Mean-Field-Approximation

Mean-Field Methoden [22] approximieren die paarweisen Interaktionen eines Agenten mit allen anderen durch die Interaktion mit einer einzigen *mittleren Aktion* der Nachbarschaft und wurden in 2.3.4 eingeführt. Aus Sicht der verwandten Arbeiten ist Mean-Field-MARL die strukturell ähnlichste Alternative zu Mean-Embeddings. Beide ersetzen die explizite Modellierung aller Interaktionen durch eine kardinalitätsunabhängige, aggregierte Statistik.

Der Unterschied liegt in zwei Punkten. Erstens aggregieren Mean-Field Verfahren im *Ursprünglichen Aktionsraum*, während Mean-Embeddings die Observations über ein gelerntes φ -Netzwerk in einen adaptiven Merkmalsraum projizieren und dort mitteln. Zweitens liegt der Aggregationspunkt bei Mean-Field auf den *Aktionen* der Nachbarn, wohingegen Mean-Embeddings auf den *Observations* aus Sicht des Agenten liegt.

Aktuellere Arbeiten erweitern den Ansatz der Mean-Fields auf kontinuierliche Schwarmkontrolle mit Kollisionsvermeidung [47] und auf gemischte Major-Minor Populationen [48]. Diese Arbeiten validieren die generelle Brauchbarkeit aggregierter Statistiken für stark skalierende MARL-Pipelines, adressieren dabei aber nicht die Dimension des gelernten Merkmalsraums, da dieser bei Mean-Field Verfahren aufgrund der Architektur fehlt.

3.1.5 Einordnung von Mean-Embeddings zu anderen Architekturen

Die diskutierten Architekturen lassen sich in einem 2-Dimensionalen Raum einordnen. Eine der Achsen drückt aus *wo* die Invarianz wirkt (Observation- vs. Critic-seitig), die andere drückt aus *wie ausdrucksstark* der Aggregationsmechanismus ist (starr vs. adaptiv).

- PIC [41] macht den Critic invariant, lässt den Actor aber unverändert und nutzt eine starre Graph Convolutional Network (GCN)-Max-Aggregation.
- GNNs [43] aggregieren auf Seite der Observations des Actors, modellieren Interaktionen allerdings mehrstufig und lokal.
- Attention [45], [46] aggregiert adaptiv mit datenabhängigen Gewichten und damit höherer Ausdruckskraft, aber quadratischer Komplexität.
- Mean-Field [22] aggregiert kardinalitätsunabhängig im Aktionsraum, ohne gelernten Merkmalsraum.
- Mean-Embeddings nach Hüttenrauch, Šošić und Neumann [6] aggregieren auf der Beobachtungsseite des Actors mit einem starren Mittelwert über ein gelerntes φ -Netzwerk.

Mean-Embeddings besetzen damit den Punkt minimaler Aggregationskomplexität bei gleichzeitig gelerntem, aufgabenadaptivem Merkmalsraum. Diese Position macht sie theoretisch zur sparsamsten konsistenten Realisierung der in 2.4.3 genannten Anforderungen und damit zum geeigneten Untersuchungsobjekt, wenn die Frage nach der *minimal notwendigen Repräsentationskapazität* im Vordergrund steht. Genau diese Frage adressiert der folgende Abschnitt.

3.2 Dimensionsreduktion und Repräsentationskapazität

Die in 3.1 genannten Arbeiten adressieren die strukturelle Frage, wie eine permutations- und kardinalitätsinvariante Aggregation aussieht. Sie lassen die Frage offen, welche Dimension der Embedding Vektor haben muss. Die in 2.6 eingeführt Schranke $k \geq M$ nach Wagstaff, Fuchs, Engelcke u. a. [34] liefert eine Worst-Case Antwort, die auch von den Autoren selbst als deutlich pessimistisch für konkrete

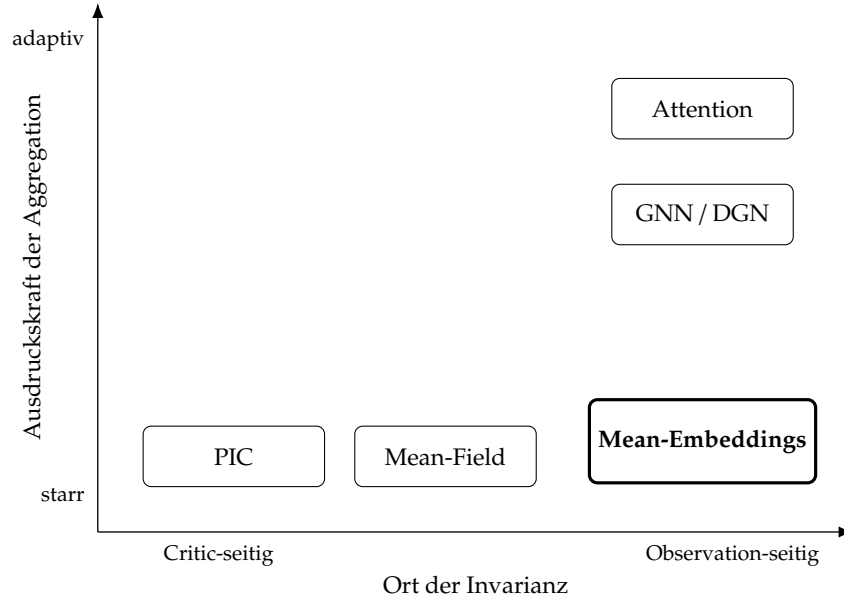


Abbildung 5: Einordnung der diskutierten Architekturen entlang des Orts der Invarianz (Critic- vs. Observation-seitig) und der Ausdrucksstärke der Aggregation (starr vs. adaptiv). Mean-Embeddings nach Hüttenrauch, Šošić und Neumann [6] besetzen den Punkt minimaler Aggregationskomplexität bei gleichzeitig gelerntem, aufgabenadaptivem Merkmalsraum.

Aufgaben gezeichnet wird. Der vorliegende Abschnitt motiviert eine empirische Untersuchung der tatsächlich notwendigen Dimension auf zwei Arten. Die Information-Bottleneck-Theorie und die Anwendungen dieser auf Repräsentationslernen in (Multi-Agent) Reinforcement Learning.

3.2.1 Information-Bottleneck-Theorie

Das Information-Bottleneck Prinzip [49] formalisiert Repräsentationslernen als Trade-off zwischen Kompression der Eingabe und Erhaltung der für die Zielvariable relevanten Informationen. Für eine Eingabe X , ein Ziel Y und eine Zwischendarstellung T ist das Lagrange-Optimierungsproblem

$$\min_{p(t|x)} I(X;T) - \beta I(T;Y) \quad (53)$$

formuliert, wobei $\beta > 0$ den Trade-off steuert. Tishby, Pereira und Bialek [49] übertragen das Prinzip auf tiefere neuronale Netze, in denen jede Schicht T_ℓ eine zunehmend komprimierte Version der Eingabe darstellt $X \rightarrow T_1 \rightarrow T_2 \rightarrow \dots \rightarrow Y$. Eine kontrollierte Informationsreduktion ist in dieser Sicht kein Defekt, sondern Ziel des Trainings.

Übertragen auf den Mean-Embedding-Vektor ist

$$\bar{\varphi}_\theta(O^i) = \frac{1}{|O^i|} \sum_{o^{i,j} \in O^i} \varphi_\theta(o^{i,j}) \in \mathbb{R}^k \quad (54)$$

ein deterministisches Bottleneck zwischen der Nachbarschaftsmenge O^i und dem Policy-Output. Die Dimension k bestimmt die maximale Informationskapazität dieser Darstellung und damit eine harte Obergrenze für $I(T;Y)$. Die hauptsächliche Forschungsfrage, wie klein k sein kann ohne dass die Policy-Performance einbricht, lässt sich damit als Frage nach dem praktischen Optimum von (53) im Schwarm Reinforcement Learning interpretieren.

3.2.2 Information-Bottleneck in (Multi-Agent) RL

Goyal, Islam, Strouse u. a. [50] übertragen in *InfoBot* das Information-Bottleneck (IB)-Prinzip direkt auf den RL Kontext. Durch einen IB-Term auf einem zielkonditionierten Policy-Encoder identifizieren sie sogenannte *decision states* und zeigen eine Verbesserung in Transfer- und Explorationseigenschaften.

Pei und Hou [51] variieren diesen Ansatz mit einer Stein-variationalen Schätzung der gemeinsamen Information und beobachten, dass das von Tishby und Zaslavsky [52] beschriebene Extraktions-Kompressions-Phänomen auch in Deep RL-Agenten auftritt.

Für den MARL-Fall ist Jin, Wei, Yuan u. a. [53] direkt relevant, da diese das IB-Prinzip explizit auf die Repräsentation des Verhaltens anderer Agenten anwendet und eine niedrigdimensionale Encoder Abbildung, die hinreichend informativ für die Kooperationsentscheidung des aktuellen Agenten ist, sucht. Empirisch berichten die Autoren über eine schnellere Konvergenz und höhere Endperformance gegenüber impliziten und nicht-IB-regularisierten Repräsentationen.

Diese Arbeiten zeigen, dass das IB-Prinzip in MARL nicht nur theoretisch motiviert, sondern auch empirisch wirksam ist. Außerdem wird aufgeführt, dass die Kontrolle der Repräsentationsdimension ein praktisch hebelnder Parameter ist. Beide Befunde stützen die Hypothese, dass die in Hüttenrauch, Šošić und Neumann [6] heuristisch gewählte Dimension $k = 64$ nicht durch theoretische Notwendigkeit, sondern durch fehlende systematische Untersuchung motiviert ist.

3.2.3 Empirische Forschung zur Embedding-Dimension

Drei empirische Beobachtungslinien sind für die Wahl der Embedding-Dimension relevant.

Erstens berichten Wagstaff, Fuchs, Engelcke u. a. [54] im Median-Estimator Toy-Experiment einen *kritischen Punkt* der latenten Dimension, ab dem die Modellgüte saturiert. Dieser Punkt liegt typischerweise *unter* der theoretischen Worst-Case Schranke M und skaliert ungeführt linear mit M mit, was als Hinweis drauf gewertet werden kann, dass reale Aufgaben Struktur in den Eingaben besitzen, die die Worst-Case Analyse ignoriert.

Zweitens zeigen Frankle und Carbin [55] im Rahmen der *Lottery-Ticket*-Hypothese, dass dicht überparametrisierte, zufällig initialisierte Feed-Forward Netzwerke sparse Sub-Netze enthalten, die mit ihrer ursprünglichen Initialisierung isoliert trainiert vergleichbare Test-Accuracy erreichen. Die Aussage ist explizit nicht, dass beliebige Sub-Netze ausreichen, sondern dass für überparametrisierte Netze *einige* deutlich kleinere Architekturen prinzipiell hinreichend sind. Übertragen auf Mean-Embeddings legt dies nahe, dass auch dort eine deutlich kleinere als die heuristisch gewählte Dimension ausreichend sein könnte, ohne dass dies bei beliebiger Reduktion garantiert ist. [56] verstärken dieses Bild noch, indem sie zeigen, dass die *intrinsische Dimension* der Zielfunktion in vielen Lernproblemen deutlich unter der Parameterzahl der eingesetzten Netze liegt.

Drittens berichten Hüttenrauch, Šošić und Neumann [6] in ihrer Arbeit selbst zur Layer-Größe des φ -Netzwerks (RENDEZVOUS, 20 Agenten), dass zwischen $k = 32$ und $k = 128$ kein systematischer Unterschied im mittleren Return auftritt. Die Wahl $k = 64$ in der Originalarbeit ist damit empirisch nicht eindeutig gerechtfertigt, da die Studie der Autoren nur eine Aufgabe mit einer Schwarmgröße und einen schmalen Dimensionsbereich abdeckt.

3.3 Einordnung und Forschungslücke

Aus den vorangegangenen Abschnitten ergibt sich das folgende Bild. Mean-Embeddings sind eine strukturell sparsame Architektur, die alle Anforderungen aus 2.4.3 erfüllt. Konkurrierende Ansätze (wie z.B. PIC, DGN, MAAC oder Mean-Field) erhöhen entweder die Ausdruckskraft oder verschieben den Aggregationspunkt, beantworten aber die Frage nach der notwendigen Repräsentationsdimension nicht. Die Information-Bottleneck (IB) Theorie liefert einen formalen Rahmen für diese Frage und ist über InfoBot [50] und Information-Bottleneck-based Other agents' behavior Representation learning for Multi-agent reinforcement learning (IBORM) [53] bereits erfolgreich auf MARL übertragen worden. Die theoretische Untergrenze nach Wagstaff, Fuchs, Engelcke u. a. [54] ist scharf für beliebige stetige Funktionen, wird in ihrem eigenen Toy-Experiment allerdings deutlich unterschritten.

Für Mean-Embedding Architekturen nach Hüttenrauch, Šošić und Neumann [6] steht eine systematische Untersuchung der Auswirkungen verschiedener Embedding-Dimensionen auf Lernverhalten, Stabilität und Generalisierung in Schwarmsystemen auf Lernverhalten, Stabilität und Generalisierung in Schwarmsystemen bislang aus. Die einzige verfügbare Datenpunktreihe aus der Ablationsstudie in [6] deckt nur eine einzelne Aufgabe und Schwarmgröße ab und variiert k nur in einem schmalen Bereich. Insbesondere fehlen Aussagen über

1. Die Existenz und Lage eines aufgaben- und schwarmgrößenabhängigen kritischen Punktes unterhalb von M

2. Die Skalierung der minimal notwendigen Dimension mit der Schwarmgröße
3. Den Einfluss der Architektur des ϕ -Netzwerkes auf diese Schranke und
4. Das Generalisierungsverhalten kleiner Embedding-Dimensionen beim Transfer von Trainings auf Test-Schwarmgrößen

Diese vier Lücken bilden den empirischen Untersuchungsgegenstand der folgenden Kapitel.

4 Implementierung

Die Abschnitte 2 und 3 haben die Mean-Embedding Architektur theoretisch hergeleitet (49) und im Spektrum der skalierungsinvarianten MARL-Ansätze eingeordnet. Das vorliegende Kapitel schließt die Lücke zwischen der abstrakten Theorie und einer lauffähigen, kontrolliert variierbaren Realisierung. Im Mittelpunkt steht nicht die Wiederholung der Theorie, sondern die konkrete Frage, wie die in 2.4.3 Voraussetzungen und die in 2.5 aufgezeigten Formeln in eine praktischen Implementierung mit einer frei einstellbaren Embedding-Dimension umgesetzt wird.

4.1 Architekturentscheidungen

Die Implementierung ist eine Neuimplementierung der Architektur von Hüttenrauch, Šošić und Neumann [6] auf einem modernen, breit verwendeten Software-Stack. Während die Originalarbeit auf TRPO aufsetzt, verwendet die vorliegende Arbeit PPO [15] als Primäralgorithmus und TRPO als Vergleichsverfahren. Diese Wahl folgt der Beobachtung, dass PPO bei vergleichbarer performance deutlich weniger rechenintensiv und besser parallelisierbar ist, und dass es sich als De-Facto Standard für On-Policy Verfahren etabliert hat.

Die Umgebungen sind nach der PettingZoo ParallelEnv Schnittstelle [25] modelliert. Das Training erfolgt währenddessen durch StableBaselines3 [13]. Beide Bibliotheken sind als feste Abhängigkeit gesetzt. Der bewusste Verzicht auf die Originalcodebasis dient zwei Zielen. Erstens der Reproduzierbarkeit auf aktueller, gepflegter Infrastruktur, und zweitens der für diese Arbeit zentralen Konfigurierbarkeit der Embedding-Dimension und der ϕ -Architektur, die in der Originalimplementierung nicht standardmäßig als systematisch variierbare Variable vorgesehen ist.

Eine zentrale Entwurfsentscheidung betrifft die Granularität des Feature-Extractors. Dieser operiert auf der Beobachtung eines *einzelnen* Agenten und nicht über den konkatenierten Gesamtobservationen die der Agent über seine Nachbarschaft (In einigen Fällen auch die globale) macht. Bereits diese Dezentralisierung ist die eigentliche Grundlage der guten Skalierungsmöglichkeit, denn eine Policy, die immer den vollständigen, mit der Agentenanzahl wachsenden Observationsvektor als Eingabe erhält, wäre an eine feste Schwarmgröße gebunden. Stattdessen besitzt die Beobachtung jedes Agenten eine feste Struktur (lokale Observation, $M_{\max}$ Nachbar-Slots, Maske) und damit eine von der Agentenzahl unabhängige Dimension.

Innerhalb dieser per-Agent-Beobachtung bewirkt die Mittelwertaggregation die Unabhängigkeit von der konkreten Nachbarzahl. Sie bildet die variabel besetzte, konkatenierte Nachbarmenge auf eine abstrahierte, festdimensionale Repräsentation in $\mathbb{R}^k$ ab. Da der Mittelwert für jede nicht-leere Nachbarmenge definiert ist, bleibt diese Repräsentation auch dann wohldefiniert, wenn sich die Nachbarschaft ändert (Beispielsweise wenn ein Nachbar ausfällt oder den Kommunikationsradius verlässt). Permutationsinvarianz und Kardinalitätsunabhängigkeit sind damit unmittelbare Eigenschaften dieser Aggregation.

Das Parameter-Sharing erfüllt eine davon getrennte Funktion. Da der Schwarm homogen ist, steuert dieselbe Policy mit identischen Gewichten alle Agenten. Zusätzlich, sind die Parameter des ϕ -Netzwerkes für jeden Agenten homogen, und werden dementsprechend unter den Agenten geteilt. Dadurch lässt sich eine auf einer bestimmten Agentenzahl trainierte Policy (Und in Folge auch ϕ -Netzwerk) auf eine beliebige Schwarmgröße anwenden. Diese Eigenschaft wird in 4.3 für die Übertragung von Trainings- auf Evaluationsgrößen angewandt.

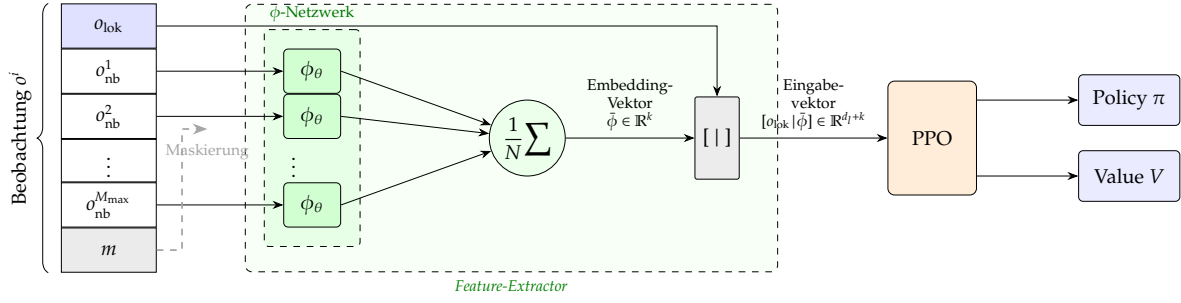


Abbildung 6: Datenfluss des Mean-Embedding-Ansatzes für einen einzelnen Agenten. Der Beobachtungsvektor o^i ist in einen lokalen Block o_{lok} , M_{max} Nachbar-Slots und eine binäre Maske m partitioniert. Die Maske entscheidet bereits am Eingang, welche der Nachbarbeobachtungen durch das über alle Nachbarn geteilte ϕ -Netzwerk verarbeitet werden. Der Mittelwert über die N aktiven Embeddings ergibt den Embedding-Vektor $\hat{\phi} \in \mathbb{R}^k$; nach Konkatination mit dem lokalen Block bildet der Eingabevektor $[o_{\text{lok}} | \hat{\phi}] \in \mathbb{R}^{d_l+k}$ die Ausgabe des Feature-Extractors. Dieser speist den Actor-Critic von PPO, der die Policy π und Wertfunktion V bereitstellt. Der gesamte Extractor wird über alle Agenten geteilt.

4.2 Mean-Embedding Feature Extractor

Der Feature-Extractor realisiert die neuronale Mittelwertaggregation aus (43) als Erweiterung der Stable-Baselines3 (SB3)-Basisklasse für Merkmalsextraktoren. Er erhält die Observation eines Agenten und erzeugt einen festdimensionalen Merkmalsvektor, der anschließend von den Policy- und Value-Netzwerken verarbeitet wird.

Beobachtungslayout Die Eingabe, also eine Observation o^i , eines Agenten ist als zusammenhängender Vektor

$$o = \left[\underbrace{o_{\text{lok}}}_{\text{lokal}} \mid \underbrace{o_{\text{nb}}}_{\text{Nachbarn}} \mid \underbrace{m}_{\text{Maske}} \right]$$

strukturiert. Der lokale block $o_{\text{lok}} \in \mathbb{R}^{d_l}$ enthält die Observation über den Agenten selbst. Der Nachbarblock besteht aus M_{max} Slots zu je d_n merkmalen, also $o_{\text{nb}} \in \mathbb{R}^{M_{\text{max}} \cdot d_n}$ und die binäre Maske $m \in \{0, 1\}^{M_{\text{max}}}$ markiert, welche Slots tatsächlich besetzte Nachbarn enthalten. Die feste Slot-Zahl M_{max} ermöglicht eine konstante maximale Eingabedimension trotz variabler, zeitlich wechselnder Nachbarschaft. Nicht belegte Slots werden mit Nullen aufgefüllt und über die Maske bereits vor der Aggregation ausgeschlossen.

ϕ -Netzwerk Das ϕ -Netzwerk bildet die Merkmale eines einzelnen Nachbarn auf den Embedding-Raum ab $\phi_\theta : \mathbb{R}^{d_n} \rightarrow \mathbb{R}^k$. Es ist als einfaches Feed-Forward Netzwerk als Folge von linearen Schichten mit nichtlinearer Aktivierungsfunktion modelliert, wobei sowohl die Breite als auch die Tiefe der verborgenen Schichten als auch die Ausgabedimension, also die Dimension des Embedding-Vektors k und die Aktivierungsfunktion konfigurierbar sind. Genau diese Konfigurierbarkeit operationalisiert die zentralen Forschungsfragen der Arbeit. Die Einstellbarkeit von k adressiert dabei die Frage nach der minimal notwendigen Embedding-Dimension, die Variation von Breite und Tiefe die Frage nach dem Einfluss der ϕ -Architektur.

Maskierte Aggregation Die Aggregation über die Nachbarn folgt der Mittelwertvorschrift aus (43). Da die Maske die Wahrnehmung ungültiger Agenten ausschließt, wird der Mittelwert ausschließlich über die $|\mathcal{N}(i)|$ validen Nachbarn gebildet.

$$\tilde{\phi}(o) = \frac{1}{\sum_j m_j} \sum_{j=1}^{M_{\text{max}}} m_j \phi_\theta(o_{\text{nb}}^j),$$

wobei der Nenner zur numerischen Stabilität nach unten beschränkt wird. Diese Form bewahrt die Permutationsinvarianz und die Kardinalitätsunabhängigkeit auch unter Padding und wechselnden Nachbarschaften. Die Mittelwertaggregation ist der Standard und entspricht der Hüttenrauch-Baseline.

Effiziente Auswertung Statt alle $M_{\max}$ Slots durch ϕ_θ zu schicken und die Padding-Beiträge nachträglich zu verwerfen werden nur die validen Nachbarzeilen extrahiert, durch das Netzwerk geführt und an ihre ursprünglichen Positionen zurückgeschrieben (*gather, process, scatter*). Bei dünn besetzten Nachbarschaften, wie sie unter lokalen Beobachtungsmodellen auftreten, vermeidet dies Rechenaufwand für leere Slots, ohne das Ergebnis zu verändern.

Ausgabe Das Aggregierte Nachbar-Embedding wird mit dem lokalen Block konkateniert, sodass der finale Merkmalsvektor die Dimension $d_l + k$ besitzt. Diese Dimension ist unabhängig von der Agentenzahl und bildet die Eingabe der nachgelagerten Policy- und Value-Netzwerke. Für den Grenzfall ohne Nachbarkomponente ($d_n = 0$) meldet der Extractor die reduzierte Merkmalsdimension d_l , sodass die Folgenetze entsprechend dimensioniert werden.

4.3 Umgebungen und Observation modes

Als Benchmark dienen zwei Schwarmaufgaben, die direkt an die Aufgaben aus [6] anschließen: *Rendezvous* und *Pursuit-Evasion*. Sie unterscheiden sich in der Struktur der Kooperation. Beide sind als PettingZoo ParallelEnv implementiert, in der Alle Agenten pro Zeitschritt simultan handeln.

Rendezvous In der Rendezvous-Aufgabe sollen die Agenten ihre paarweisen Abstände minimieren und sich an einem gemeinsamen, nicht vorgegebenen Punkt versammeln. Die geteilte Belohnung folgt der Definition aus [6, Appendix E.1] und ist über die paarweisen Abstände $d^{i,j}$ definiert

$$R(s, a) = \alpha \sum_{i=1}^N \sum_{j=i+1}^N \min(d^{i,j}, d_c) + \beta \|a\|,$$

wobei der erste Term die Summer der auf d_c , der Größe der Welt, begrenzten paarweisen Abstände bestraft und der zweite über $\beta = -10^{-3}$ die Stärke des Steuersignals abmildert, um unnötig große Stellgrößen zu vermeiden. Im global beobachteten Fall ist die Cut-off-Distanz $d_c = \max(x_{\max}, y_{\max})$ die maximal mögliche Distanz im jeweiligen Weltausschnitt. Der Normierungsfaktor $\alpha = -(\frac{N(N-1)}{2}d_c)^{-1}$ skaliert den Distanzterm so, dass der Return näherungsweise im Intervall $[-1, 0]$ liegt. Die Episoden laufen, bis eine gewisse paarweise Abstands-Grenze unterschritten ist, falls diese Grenze angegeben wird. Andernfalls wird immer die Maximale Schrittzahl erreicht. Die Aufgabe ist rein kooperativ und prüft, ob die aggregierte Repräsentation ausreicht um koordinierte Konvergenz zu erzeugen.

Pursuit-Evasion In der Pursuit-Evasion Aufgabe verfolgt ein Schwarm von Pursuern einen Evader. Der Evader ist **kein** lernender Agent sondern folgt der Voronoi-Ridge Ausweichstrategie aus [6] und ist damit Teil der Umgebungsdynamik. Die Aufgabe ist dementsprechend kooperativer Natur und nicht kompetitiv. Die Belohnung ist über alle Verfolger geteilt und folgt der Formulierung aus [6]

$$r = - \frac{\min(d_{\min}, r_{\text{obs}})}{r_{\text{obs}}},$$

mit $d_{\min}$ als kleinstem Verfolger-Evader-Abstand und r_{obs} als Beobachtungsradius. Die Episode terminiert, sobald ein Verfolger den Evader innerhalb des Fangradius erreicht.

Beobachtungsmodelle Beide Umgebungen unterstützen mehrere Beobachtungsmodelle, die sich entlang zweier Achsen unterscheiden. Die erste Achse betrifft die *Observability*. Unter globaler beobachtbarkeit nimmt jeder Agent alle übrigen Agenten wahr, unter lokaler nur Nachbarn innerhalb eines *Kommunikationsradius*. Die zweite Achse betrifft den *Informationsgehalt* pro Nachbar. In Anlehnung an [6] setzt sich eine Nachbarbeobachtung $o^{i,j}$ und der lokale Block o_{loc}^i wie folgt zusammen, wobei $d^{i,j}$ den Abstand, $\phi^{i,j}$ den Lagewinkel, $\theta^{i,j}$ die relative Orientierung und $\Delta v^{i,j}$ die relative Geschwindigkeit zum Nachbarn bezeichnet

$$\begin{aligned} \text{basic: } o^{i,j} &= \{d^{i,j}, \phi^{i,j}\} \\ \text{extended: } o^{i,j} &= \{d^{i,j}, \phi^{i,j}, \theta^{i,j}, \Delta v^{i,j}\} \end{aligned}$$

Tabelle 1: Beobachtungslayout je Modell. d_l bezeichnet die Dimension des lokalen Blocks, d_n die Merkmalszahl pro Nachbar. Lagewinkel und Orientierungen sind als (cos, sin)-Paar kodiert.

Umgebung	Modell	d_l	d_n
Rendezvous	global_basic	3	3
	local_basic	3	3
	global_extended	3	7
	local_extended	3	5
	local_comm	4	6
Pursuit-Evasion	global_basic	6	3
	local_basic	6	3
	global_extended	6	6
	local_extended	6	5

Die relative Geschwindigkeit $\Delta v^{i,j}$ ist dabei nur im global beobachtbaren Fall Teil des *extended* Modells. Lagewinkel und Orientierungen werden durchgängig als (cos, sin) Paar kodiert, um die für neuronale Netze ungünstige Sprungstelle bei $\pm\pi$ zu vermeiden. In pursuit-Evasion umfasst der lokale Block o_{loc}^i zusätzlich die Evader-Merkmale (Abstand und Lagewinkel als (cos, sin)). Der Evader wird damit dem Eigenzustand zugeordnet und nimmt nicht an der Nachbar-Aggregation teil, wodurch die Skaleninvarianz über die Verfolgerzahl erhalten bleibt. Welche Modelle in den Hauptexperimenten verwendet werden, ist in 5 festgelegt. Das Resultierende Layout fasst 1 zusammen.

Kinematik und Skalentransfer Die Agentenbewegung ist wahlweise als *single*- oder *double*- Integrator modelliert. Die Skaleninvarianz auf Implementierungsebene ergibt sich aus dem Zusammenspiel dreier Elemente

- Der Mittelwertaggregation aus 4.2
- Der festen Slot-Zahl M_{max} mit der Maskierung
- Der über alle Agenten geteilte Policy

In Kombination erlauben sie, eine Policy auf einer Schwarmgröße zu trainieren und auf einer abweichenden Schwarmgröße zu evaluieren, ohne die Netzstruktur ändern zu müssen.

4.4 Integration in Stable-Baselines3

Der Mean-Embedding Feature-Extractor wird über den Konfigurationsmechanismus von SB3 in die Lernalgorithmen eingebunden. Eine zentrale Hilfsfunktion baut aus dem Beobachtungslayout der jeweiligen Umgebung (d_l, d_n, M_{max}) und den gewünschten Hyperparametern (k, ϕ - Tiefe und -Breite, Aktivierung, Aggregation) die vollständige Extractor Architektur zusammen. Policy- und Value-Netzwerke werden über die Architekturvorgabe von SB3 getrennt parametrisiert. Sie verarbeiten den vom Extractor erzeugten Merkmalsvektor weiter. Dadurch sind der austauschbare Teil (Die Repräsentation durch den Extractor) und der aufgabenspezifische Teil (Die Policy/Value Netzwerke) sauber voneinander getrennt, können allerdings gemeinsam trainiert werden.

Die ParallelEnv-Umgebungen werden vor dem Training in die vektorisierte Form überführt, die SB3 erwartet. Auf eine laufende Beobachtungs- und Belohnungsnormalisierung (VecNormalize) wird dabei bewusst verzichtet. Dies ist durch das Observation- und Rewarddesign begründet, da hier die Abstände bereits durch die Weltgröße normiert werden, die Lagewinkel als (cos, sin) Paare bereits zwischen $[-1, 1]$ liegen, und die Belohnung nach [6] auf $[-1, 0]$ skaliert. Eine zusätzlich laufende Normalisierung würde zudem die binäre Nachbarsmaske verfälschen, die ganzzahlige Werte in $\{0, 1\}$ voraussetzt. Da Implementierungsseitige Details von On-Policy Verfahren das Ergebnis messbar beeinflussen und einen vergleich über die Embedding-Dimension sonst verzerren könnten [17], [18], ist diese Festlegung für alle Läufe identisch fixiert. PPO nutzt die generalisierte Vorteilsschätzung [11], wobei die vollständige Konfiguration der Hyperparameter einschließlich der verwendeten Werte in ?? thematisiert wird.

4.5 Analyse- und Evaluationswerkzeuge

Die in 5 definierten Experimente variieren die Embedding-Dimension und die ϕ -Architektur über mehrere Schwarmgrößen. Da die Leistung tiefer RL-Verfahren über verschiedene Seeds stark streut, sind Punktschätzer wie der Mittelwert für belastbare Aussagen unzureichend. Die Auswertung folgt daher der Methodik von Agarwal, Schwarzer, Castro u. a. [57] und nutzt deren Bibliothek. Berichtet werden der Interquartile Mean (IQM) als ausreißerrobuster und statistisch effizienter Aggregat-Estimator, stratifizierte Bootstrap-Konfidenzintervalle zur Quantifizierung der Unsicherheit sowie *Performance Profiles* zur Darstellung der Leistungsverteilung über Läufe und Aufgaben hinweg.

Die Aggregationsvarianten jenseits des Mittelwerts (Summe, Maximum, Attention) werden ausschließlich als optionale Ablation behandelt und strikt von den Hauptexperimenten ausgeschlossen, um die untersuchende Wirkung der Embedding-Dimension nicht mit dem Aggregationsmechanismus zu vermengen. Die so erzeugten Estimators und Profile bilden die Grundlage der Ergebnisdarstellung in 6.

5 Design der Experimente

Dieses Kapitel überführt die in 1.2 formulierten Forschungsfragen in ein konkretes Experimentdesign. Während 4 beschreibt, wie genau der Mean-Embedding Extractor die Umgebungen und die Auswertungswerkzeuge realisiert sind, legt dieses Kapitel fest, welche Konfigurationen trainiert werden, warum genau diese Auswahl die Forschungsfragen beantwortet und wie die Ergebnisse gemessen und ausgewertet werden. Die Darstellung folgt einem Trichter, von der Operationalisierung der Fragen über die Versuchsbedingungen und das Trainingsprotokoll bis zur Mess- und Auswertmethodik.

5.1 Operationalisierung der Forschungsfragen

Den theoretischen Bezugspunkt bildet die in 2.6 eingeführte untere Schranke $k \geq M$ nach Wagstaff, Fuchs, Engelcke u. a. [34], wobei M die Kardinalität der aggregierten Nachbarmenge (Die Agentenanzahl) und k die Dimension des Embedding-Vektors bezeichnet. Diese Schranke ist *notwendig* für die Exakte darstellung beliebiger stetiger permutationsinvarianter Funktionen, wird wie bereits erwähnt in der Praxis jedoch deutlich unterschritten. Daras ergibt sich die zentrale empirische Frage, wo für konkrete Schwarmaufgaben die tatsächlich erforderliche Embedding-Dimension liegt und wie sie sich relativ zu M verhält.

Die Tabelle 2 bildet die Forschungsfragen auf die manipulierten Variablen, für die zugehörige Experimentreihe und die zu prüfenden Hypothesen ab. Die variierenden Stellschrauben sind die Embedding-Dimension k , Die Architektur des ϕ -Netzwerkes (Tiefe L und Breite w) als auch die Schwarmgröße N . Jede Gruppe isoliert dabei genau einen Faktorblock, sodass beobachtete Effekte eindeutig zugeordnet werden können.

Tabelle 2: Zuordnung von Forschungsfragen, manipulierten Variablen, Experimentreihen und Hypothesen. Die Bezeichnung der Fragen folgt der inhaltlichen Fragestellung aus Abschnitt ??.

Forschungsfrage	Manipulierte Variable	Experimentreihe	Hypothese
RQ1: Einfluss von k	k	Embedding-Skalierung	H1
RQ2: minimale Dimension	k (untere Stufen)	Embedding-Skalierung	H1
RQ3: Skalierung mit N / Transfer	$N_{\text{train}} \rightarrow N_{\text{test}}$	Embedding-Skalierung + Transfer	H2, H3
RQ4: Einfluss der ϕ -Architektur	L, w	ϕ -Architekturvariation	H4

5.2 Aufgaben und Versuchsbedingungen

Aks Testumgebungen dienen die beiden in 4.3 eingeführten Aufgaben *Rendezvous* und *Pursuit-Evasion*. Beide stammen aus der Referenzarbeit von Hüttenrauch, Šošić und Neumann [6] und sichern damit die Vergleichbarkeit mit der ursprünglichen Mean-Embedding-Architektur. Sie fallen in die in 2.4.2 genannten Kategorien räumliche Organisation und Navigation, prüfen jedoch komplementäre

Koordinationsformen. Rendezvous verlangt rein kooperative Konvergenz auf einen gemeinsamen, nicht vorgegebenen Punkt, während Pursuit-Evasion koordiniertes Verfolgen eines nicht lernenden Evaders mit fester Evasion-Strategie erfordert. Diese Gegenüberstellung ist für H1 entscheidend, da sie eine *aufgabenunabhängige* kritische Dimension sichtbar machen kann.

Um die Wirkung der manipulierten Faktoren nicht mit weiteren Einflussgrößen zu vermischen, werden die übrigen Bedingungen über alle Hauptexperimente konstant gehalten.

Verwendet wird durchgängig das Beobachtungsmodell `global.basic`, eine Single-Integrator Kinematik und eine torusförmige Welt. Die Episodenlänge beträgt 500 Schritte für Rendezvous und 1024 Schritte für Pursuit Evasion. Als Erfolgskriterium dienen das Unterschreiten eines paarweisen Abstands von 2.0 bei Rendezvous, bzw. das Erreichen des Evaders innerhalb des Fangradiuses 2.0 bei Pursuit-Evasion. Die formale Definition von beobachtungslayout und Belohnungen ist in 4.3 vorgegeben und wird hier nicht wiederholt. Die Beschränkung auf `global.basic` grenzt den Untersuchungsraum stark ein. Lokale Beobachtbarkeit bleibt außerhalb des Hauptdesigns und wird in ?? als Grenze definiert.

5.3 Konfigurationsmatrix

Die Hauptexperimente bestehen aus zwei Trainingsreihen, sowie einem darauf aufsetzenden Transfer-Protokoll auf Evaluationsebene. Die erste Reihe, im Folgenden als *Embedding-Skalierung* bezeichnet, variiert, wie der Name vermuten lässt, die Embedding-Dimension k bei festem ϕ -Netzwerk. Die zweite Reihe, die ϕ -Architekturvariation, hält k konstant, variiert allerdings die Tiefe L und breite w des ϕ -Netzwerks. Die Tabelle 3 fasst die variierten und fixierten Größen zusammen.

Tabelle 3: Konfigurationsmatrix der Hauptexperimente. Die Stufen entsprechen den Experiment-Konfigurationen des Repositoriums. Der Embedding-Wert $k = 64$ markiert die Wahl der Referenzarbeit [hutzenrauch2019](#).

Experimentreihe	Aufgabe	Manipulierte (Stufen)	Faktoren	Fixiert
Embedding-Skalierung	Rendezvous, Pursuit-Evasion	$k \in \{4, 8, 16, 32, 64, 128\};$ $N \in \{4, 16, 50, 100\}$		$\phi: L=1, w=64;$ Aggregation: Mittelwert
ϕ -Architekturvariation	Rendezvous	$L \in \{1, 2, 4\};$ $w \in \{32, 64, 128\}$		$k=64; N=16;$ Aggregation: Mittelwert
Transfer (Evaluation)	Policies der Embedding-Skalierung	$N_{\text{test}} \geq N_{\text{train}},$ bis $N_{\text{test}} \gg N_{\text{train}}$		Policy eingefroren; deterministische Rollouts

Die Embedding-Skalierung variiert die Embedding-Dimension über sechs Stufen und vier Schwarmgrößen bei einem festen, einlagigen ϕ -Netzwerk und adressiert damit RQ1 bis RQ3 sowie H1 bis H3. Die ϕ -Architekturvariation hält $k = 64$ konstant und variiert die Tiefe und Breite des ϕ -Netzwerks und adressiert RQ4 und H4. Der Skalentransfer (RQ3) erzeugt keine zusätzlichen Trainingsläufe, sondern wertet jede in der Embedding-Skalierung trainierte Policy auf mehreren Schwarmgrößen aus. Dies nutzt die Skaleninvarianz der Architektur 4.2 und trennt Trainings- und Evaluationsgrößen. Damit der Transfer auf Schwärme oberhalb der Trainingsgröße möglich ist, wird die maximale Slotzahl M_{max} auf die jeweils *größte evaluierte* Schwarmgröße dimensioniert, nicht auf die Trainingsgröße. Eine reine Mittelwertaggregation wird in beiden Trainingsreihen verwendet. Alternative Aggregationen (Summe, Maximum, Attention) bleiben gemäß ?? auf rein optionale Ablation beschränkt und sind von den Hauptexperimenten ausgeschlossen.

5.4 Trainingsprotokoll und Hyperparameter

Alle Konfigurationen werden mit PPO [15] unter generalisierter Vorteilsschätzung [11] trainiert. Die Implementierung erfolgt über Stable-Baselines3 [13] auf den als PettingZoo-Umgebungen realisierten [25] Aufgaben (Unterabschnitt 4.4). Die Wahl von PPO als genutzten Optimierungsalgorithmus ist

Tabelle 4: Trainings-Hyperparameter. Abweichende Werte je Aufgabe sind getrennt angegeben.

Hyperparameter	Rendezvous	Pursuit-Evasion
Algorithmus / Clipping ϵ	PPO / 0,2	PPO / 0,2
Lernrate (Adam)	$3 \cdot 10^{-4}$	$3 \cdot 10^{-4}$
Rollout-Länge T (n.steps)	500	500
Minibatch-Größe	500	500
Epochen je Update	4	4
Ziel-KL	0,015	0,015
Diskont γ / GAE λ	0,99/0,98	0,99/0,98
Politik-Updates I (Iterationen)	500	1000
Parallele Umgebungen E	8 ($N=4$), 4 ($N=16$), 1 ($N \geq 50$)	8 ($N=4$), 4 ($N=16$), 1 ($N \geq 50$)
ϕ -Aktivierung / Aggregation	ReLU / Mittelwert	ReLU / Mittelwert
Policy-/Value-Kopf	je ein Layer, Breite 64	je ein Layer, Breite 64

eine bewusste Abweichung der Referenzarbeit [6], welche **TRPO** einsetzt. Jegliche Vergleiche zu [6] sind daher nicht auf den Optimierungs-Algorithmus, sondern auf die Repräsentations-Architektur zu beziehen. Da **PPO** sehr empfindlich auf Implementierungs- und Hyperparameterentscheidungen reagiert [17], [18], werden sämtliche Hyperparameter über alle Konfigurationen einer Trainingsreihe konstant gehalten. Neben den Faktoren aus **Tabelle 3** wird nur die Zahl paralleler Umgebungen E , aufgrund von Laufzeitgründen für verschiedene Schwarmgrößen, variiert. Der Discount-Faktor und die **GAE**-Parameter werden an [6] angelehnt.

Das Trainingsbudget wird nicht über die Anzahl der Umgebungsschritte, sondern über die Policy-Updates definiert. Bei I Iterationen, Rollout-Länge T und Schwarmgröße N und E parallelen Environments ergibt sich die Gesamtzahl der Übergänge aus

$$T_{\text{total}} = I \cdot T \cdot N \cdot E. \quad (55)$$

Die Zahl der Policy-Updates wird konstant gehalten, sodass die Zahl der Gradient-Updates, und damit Lernsignal pro Agent, über die Schwarmgrößen vergleichbar bleibt. Dies entspricht der iterationsbasierten Darstellung in [6]. Die Anzahl paralleler Environments wird hingegen aus Laufzeitgründen für kleine Schwärme erhöht. Aus **Gleichung 55** folgt damit, dass T_{total} sowohl mit N , als auch mit dem an N gekoppelten Faktor E variiert. Diese Kopplung an die Schwarmgröße wird in ?? als möglicher Störfaktor der Skalierungsaussagen diskutiert. Je Konfiguration wird T_{total} daher mitbeachtet.

Aus **Gleichung 55** folgt allerdings, dass T_{total} mit N wächst. Diese verbleibende Kopplung wird in ?? als möglicher Störfaktor der Skalierungsaussagen diskutiert. Je Konfiguration wird T_{total} daher mitberichtet.

Das Training wird über TensorBoard mithilfe der Logging-Mechanismen von Stable-Baselines3 protokolliert. Dabei werden der mittlere und der Streuungswert der episoden Rewards (rollout/ep_rew_mean, ep_rew_std), die Episodenlänge, der Durchsatz in Transitions pro Sekunde sowie die aufgabenspezifischen Kennzahlen "Erfolgsrate" (task/success_rate), Konvergenzgeschwindigkeit und Fangzeit. Sämtliche Metriken werden gegen die Zahl der Umgebungsschritte und die Iterationszahl (by_iter/) geloggt, um Lernkurven über Schwarmgrößen hinweg iterationsweise vergleichbar zu machen.

5.5 Evaluations- und Metrikprotokoll

Nachdem das Training abgeschlossen ist, wird jede Policy auf festen Mengen von Schwarmgrößen über jeweils 100 Episoden mit deterministischer Aktionswahl evaluiert. Die deterministische Auswertung misst das Verhalten der Policy und entkoppelt die Bewertung von der Explorationsstreuung. Als primäre Leistungsgröße dient der mittlere episodische Return $\bar{G}$. Ergänzend werden je Aufgabe interpretierbare Kennzahlen erhoben:

Rendezvous

- Erfolgsrate
- Finaler mittlerer paarweiser Abstand

- Finaler maximaler paarweiser Abstand
- Distanz zum Schwarmsschwerpunkt

Pursuit-Evasion

- Fangrate
- Fangzeit

Die Generalisierung über verschiedene Schwarmgrößen wird über den Generalisierungs-Abstand quantifiziert. Für eine auf N_{train} trainierte Policy ist dieser bei Testgröße N_{test} definiert als

$$\Delta(N_{\text{test}}) = \bar{G}(N_{\text{train}}) - \bar{G}(N_{\text{test}}), \quad (56)$$

wobei kleinere Werte auf eine bessere Übertragbarkeit hinweisen. Da die Returns der beiden auf unterschiedlichen Skalen liegen, werden sie vor der Aufgaben- und größenübergreifenden Aggregation je Zelle (Aufgabe, N) über die beobachtete Spannweite auf $[0, 1]$ normiert.

$$\tilde{G} = \frac{G - G_{\min}}{G_{\max} - G_{\min}}, \quad (57)$$

sodass die in [Unterabschnitt 5.6](#) verwendeten Estimator und Profile auf einer einheitlichen Skala agieren.

5.6 Statistische Auswertung

Die statistische Auswertung folgt der in [Unterabschnitt 4.5](#) eingeführten Methodik nach Agarwal, Schwarzer, Castro u. a. [57] und wird hier auf das Experimentdesign angewandt. Da die Performance von Deep-RL-Verfahren über zufälliges Seeding stark streuen kann und punktuelle Schätzungen aus wenigen Läufen nicht belastbar sind [58], bildet die Menge der je Konfiguration trainierten Seeds die für die Aggregation maßgebliche Run-Dimension. Pro Konfiguration werden S unabhängige Seeds mit den Werten $0, \dots, S - 1$ trainiert. Die hier dargelegte Methodik adressiert genau das Problem weniger Läufe und erfordert mehrere Seeds, nicht einen einzelnen Lauf [57].

Berichtet werden der IQM als ausreisserrobuster und statistisch effizienter Schätzer, 95 %-Konfidenzintervalle über stratifizierte Bootstrap-Stichproben sowie Performance Profiles als Lauf-Score-Verteilungen über Konfigurationen, Aufgaben und Schwarmgrößen hinweg. Die nach [Gleichung 57](#) normierten Returns bilden die Eingabe für diese Estimator. Auf Basis dieser Estimators und Profile erfolgt die Ergebnisdarstellung in [Abschnitt 6](#)

6 Ergebnisse

6.1 Skalierbarkeit des Embedding-Vektors

6.2 Skalierbarkeit des Phi-Networks

6.3 Zusammenfassung

7 Diskussion

7.1 Interpretation der Ergebnisse

7.2 Implikationen für Swarm-RL

7.3 Vergleich mit ähnlichen Arbeiten

7.4 Grenzen

8 Fazit

8.1 Zusammenfassung

8.2 Beantwortung der Hypothesen

8.3 Zukünftige Arbeit

Literatur

- [1] R. S. Sutton und A. G. Barto, *Reinforcement Learning: An Introduction*, 2. Aufl. Cambridge, MA: MIT Press, 2018, ISBN: 9780262039246.
- [2] V. Mnih, K. Kavukcuoglu, D. Silver u. a., „Human-level control through deep reinforcement learning,“ *Nature*, Jg. 518, Nr. 7540, S. 529–533, 2015. doi: [10.1038/nature14236](https://doi.org/10.1038/nature14236). Adresse: <https://www.nature.com/articles/nature14236>.
- [3] M. Brambilla, E. Ferrante, M. Birattari und M. Dorigo, „Swarm robotics: a review from the swarm engineering perspective,“ *Swarm Intelligence*, Jg. 7, Nr. 1, S. 1–41, 2013. doi: [10.1007/s11721-012-0075-2](https://doi.org/10.1007/s11721-012-0075-2).
- [4] M. Schranz, M. Umlauft, M. Sende und W. Elmenreich, „Swarm Robotic Behaviors and Current Applications,“ *Frontiers in Robotics and AI*, Jg. 7, S. 36, 2020. doi: [10.3389/frobt.2020.00036](https://doi.org/10.3389/frobt.2020.00036).
- [5] P. Hernandez-Leal, B. Kartal und M. E. Taylor, „A Survey and Critique of Multiagent Deep Reinforcement Learning,“ *Autonomous Agents and Multi-Agent Systems*, Jg. 33, Nr. 6, S. 750–797, 2019. doi: [10.1007/s10458-019-09421-1](https://doi.org/10.1007/s10458-019-09421-1).
- [6] M. Hüttenrauch, A. Šošić und G. Neumann, *Deep Reinforcement Learning for Swarm Systems*, 2019. arXiv: [1807.06613](https://arxiv.org/abs/1807.06613) [cs.MA]. Adresse: <https://arxiv.org/abs/1807.06613>.
- [7] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel und I. Mordatch, „Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments,“ in *Advances in Neural Information Processing Systems (NeurIPS)* 30, 2017, S. 6379–6390. arXiv: [1706.02275](https://arxiv.org/abs/1706.02275).
- [8] R. S. Sutton, D. McAllester, S. Singh und Y. Mansour, „Policy Gradient Methods for Reinforcement Learning with Function Approximation,“ in *Advances in Neural Information Processing Systems*, Bd. 12, 2000, S. 1057–1063.
- [9] R. J. Williams, „Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning,“ *Machine Learning*, Jg. 8, Nr. 3–4, S. 229–256, 1992. doi: [10.1007/BF00992696](https://doi.org/10.1007/BF00992696).
- [10] V. R. Konda und J. N. Tsitsiklis, „Actor-Critic Algorithms,“ in *Advances in Neural Information Processing Systems*, Bd. 12, 2000, S. 1008–1014.
- [11] J. Schulman, P. Moritz, S. Levine, M. I. Jordan und P. Abbeel, „High-Dimensional Continuous Control Using Generalized Advantage Estimation,“ in *International Conference on Learning Representations (ICLR)*, 2016. arXiv: [1506.02438](https://arxiv.org/abs/1506.02438) [cs.LG].
- [12] V. Mnih, A. P. Badia, M. Mirza u. a., „Asynchronous Methods for Deep Reinforcement Learning,“ in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016, S. 1928–1937. arXiv: [1602.01783](https://arxiv.org/abs/1602.01783).
- [13] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus und N. Dormann, „Stable-Baselines3: Reliable Reinforcement Learning Implementations,“ *Journal of Machine Learning Research*, Jg. 22, Nr. 268, S. 1–8, 2021.
- [14] J. Schulman, S. Levine, P. Moritz, M. I. Jordan und P. Abbeel, „Trust Region Policy Optimization,“ in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015, S. 1889–1897. arXiv: [1502.05477](https://arxiv.org/abs/1502.05477).
- [15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford und O. Klimov, *Proximal Policy Optimization Algorithms*, 2017. arXiv: [1707.06347](https://arxiv.org/abs/1707.06347).
- [16] S. M. Kakade, „A Natural Policy Gradient,“ in *Advances in Neural Information Processing Systems*, Bd. 14, 2002, S. 1531–1538.
- [17] L. Engstrom, A. Ilyas, S. Santurkar u. a., „Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO,“ in *International Conference on Learning Representations (ICLR)*, 2020. arXiv: [2005.12729](https://arxiv.org/abs/2005.12729).
- [18] M. Andrychowicz, A. Raichuk, P. Stańczyk u. a., *What Matters in On-Policy Reinforcement Learning? A Large-Scale Empirical Study*, 2021. arXiv: [2006.05990](https://arxiv.org/abs/2006.05990).
- [19] S. V. Albrecht, F. Christianos und L. Schäfer, *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. Cambridge, MA: MIT Press, 2024, ISBN: 9780262049375.

- [20] S. Gronauer und K. Diepold, „Multi-Agent Deep Reinforcement Learning: A Survey,“ *Artificial Intelligence Review*, Jg. 55, Nr. 2, S. 895–943, 2022. doi: [10.1007/s10462-021-09996-w](https://doi.org/10.1007/s10462-021-09996-w).
- [21] C. Yu, A. Velu, E. Vinitsky u. a., „The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games,“ in *Advances in Neural Information Processing Systems (NeurIPS)* 35, 2022. arXiv: [2103.01955](https://arxiv.org/abs/2103.01955).
- [22] Y. Yang, R. Luo, M. Li, M. Zhou, W. Zhang und J. Wang, „Mean Field Multi-Agent Reinforcement Learning,“ in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, S. 5567–5576. arXiv: [1802.05438](https://arxiv.org/abs/1802.05438).
- [23] M. L. Littman, „Markov Games as a Framework for Multi-Agent Reinforcement Learning,“ in *Proceedings of the 11th International Conference on Machine Learning (ICML)*, Morgan Kaufmann, 1994, S. 157–163. doi: [10.1016/B978-1-55860-335-6.50027-1](https://doi.org/10.1016/B978-1-55860-335-6.50027-1).
- [24] D. S. Bernstein, S. Zilberstein und N. Immerman, „The Complexity of Decentralized Control of Markov Decision Processes,“ 2013. arXiv: [1301.3836 \[cs.AI\]](https://arxiv.org/abs/1301.3836). Adresse: <https://arxiv.org/abs/1301.3836>.
- [25] J. K. Terry, B. Black, N. Grammel u. a., „PettingZoo: Gym for Multi-Agent Reinforcement Learning,“ in *Advances in Neural Information Processing Systems (NeurIPS)* 34, 2021, S. 15 032–15 043. arXiv: [2009.14471](https://arxiv.org/abs/2009.14471).
- [26] P. Sunehag, G. Lever, A. Gruslys u. a., „Value-Decomposition Networks For Cooperative Multi-Agent Learning Based On Team Reward,“ in *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, 2018, S. 2085–2087. arXiv: [1706.05296](https://arxiv.org/abs/1706.05296).
- [27] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster und S. Whiteson, „QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning,“ 2018. arXiv: [1803.11485 \[cs.LG\]](https://arxiv.org/abs/1803.11485). Adresse: <https://arxiv.org/abs/1803.11485>.
- [28] J. N. Foerster, G. Farquhar, T. Afouras, N. Nardelli und S. Whiteson, „Counterfactual Multi-Agent Policy Gradients,“ in *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, 2018. arXiv: [1705.08926](https://arxiv.org/abs/1705.08926).
- [29] J. K. Terry, N. Grammel, S. Son, B. Black und A. Agrawal, „Revisiting Parameter Sharing in Multi-Agent Deep Reinforcement Learning,“ *arXiv preprint arXiv:2005.13625*, 2021.
- [30] J. K. Gupta, M. Egorov und M. J. Kochenderfer, „Cooperative Multi-Agent Control Using Deep Reinforcement Learning,“ in *Autonomous Agents and Multiagent Systems (AAMAS) Workshops, Lecture Notes in Computer Science*, Bd. 10642, Springer, 2017, S. 66–83. doi: [10.1007/978-3-319-71682-4_5](https://doi.org/10.1007/978-3-319-71682-4_5).
- [31] E. Şahin, „Swarm Robotics: From Sources of Inspiration to Domains of Application,“ in *Swarm Robotics: SAB 2004 International Workshop, Lecture Notes in Computer Science*, Bd. 3342, Springer, 2005, S. 10–20. doi: [10.1007/978-3-540-30552-1_2](https://doi.org/10.1007/978-3-540-30552-1_2).
- [32] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov und A. Smola, „Deep Sets,“ in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. Adresse: <https://arxiv.org/abs/1703.06114>.
- [33] A. Smola, A. Gretton, L. Song und B. Schölkopf, „A Hilbert Space Embedding for Distributions,“ in *Algorithmic Learning Theory (ALT)*, Ser. Lecture Notes in Artificial Intelligence, Bd. 4754, Springer, 2007, S. 13–31. doi: [10.1007/978-3-540-75225-7_5](https://doi.org/10.1007/978-3-540-75225-7_5).
- [34] E. Wagstaff, F. Fuchs, M. Engelcke, I. Posner und M. Osborne, „On the Limitations of Representing Functions on Sets,“ in *International Conference on Machine Learning (ICML)*, 2019, S. 6487–6494. Adresse: <https://arxiv.org/abs/1901.09006>.
- [35] N. Aronszajn, „Theory of Reproducing Kernels,“ *Transactions of the American Mathematical Society*, Jg. 68, Nr. 3, S. 337–404, 1950. doi: [10.1090/S0002-9947-1950-0051437-7](https://doi.org/10.1090/S0002-9947-1950-0051437-7).
- [36] B. Schölkopf und A. J. Smola, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. Cambridge, MA: MIT Press, 2002, ISBN: 9780262194754.
- [37] T. Hofmann, B. Schölkopf und A. J. Smola, „Kernel Methods in Machine Learning,“ *The Annals of Statistics*, Jg. 36, Nr. 3, S. 1171–1220, 2008. doi: [10.1214/009053607000000677](https://doi.org/10.1214/009053607000000677).

- [38] B. K. Sriperumbudur, A. Gretton, K. Fukumizu, B. Schölkopf und G. R. G. Lanckriet, „Hilbert Space Embeddings and Metrics on Probability Measures,” *Journal of Machine Learning Research*, Jg. 11, S. 1517–1561, 2010. Adresse: <https://arxiv.org/abs/0907.5309>.
- [39] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf und A. Smola, „A Kernel Two-Sample Test,” *Journal of Machine Learning Research*, Jg. 13, Nr. 25, S. 723–773, 2012.
- [40] A. Jacot, F. Gabriel und C. Hongler, „Neural Tangent Kernel: Convergence and Generalization in Neural Networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. arXiv: [1806.07572](https://arxiv.org/abs/1806.07572).
- [41] I.-J. Liu, R. A. Yeh und A. G. Schwing, „PIC: Permutation Invariant Critic for Multi-Agent Deep Reinforcement Learning,” in *Conference on Robot Learning (CoRL)*, 2019, S. 590–602. Adresse: <https://arxiv.org/abs/1911.00025>.
- [42] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals und G. E. Dahl, *Neural Message Passing for Quantum Chemistry*, 2017. arXiv: [1704.01212](https://arxiv.org/abs/1704.01212) [cs.LG]. Adresse: <https://arxiv.org/abs/1704.01212>.
- [43] J. Jiang, C. Dun, T. Huang und Z. Lu, „Graph Convolutional Reinforcement Learning,” in *International Conference on Learning Representations (ICLR)*, 2020. Adresse: <https://arxiv.org/abs/1810.09202>.
- [44] A. Vaswani, N. Shazeer, N. Parmar u. a., *Attention Is All You Need*, 2017. arXiv: [1706.03762](https://arxiv.org/abs/1706.03762) [cs.CL]. Adresse: <https://arxiv.org/abs/1706.03762>.
- [45] J. Lee, Y. Lee, J. Kim, A. Kosiosek, S. Choi und Y. W. Teh, „Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks,” in *International Conference on Machine Learning (ICML)*, 2019, S. 3744–3753. Adresse: <https://arxiv.org/abs/1810.00825>.
- [46] S. Iqbal und F. Sha, „Actor-Attention-Critic for Multi-Agent Reinforcement Learning,” in *International Conference on Machine Learning (ICML)*, 2019, S. 2961–2970. Adresse: <https://arxiv.org/abs/1810.02912>.
- [47] K. Cui, M. Li, C. Fabian und H. Koepl, „Scalable Task-Driven Robotic Swarm Control via Collision Avoidance and Learning Mean-Field Control,” *arXiv preprint arXiv:2209.07420*, 2022.
- [48] K. Cui, C. Fabian, A. Tahir und H. Koepl, „Major-Minor Mean Field Multi-Agent Reinforcement Learning,” in *Proceedings of the 41st International Conference on Machine Learning*, R. Salakhutdinov, Z. Kolter, K. Heller u. a., Hrsg., Ser. Proceedings of Machine Learning Research, Bd. 235, PMLR, Juli 2024, S. 9603–9632. Adresse: <https://proceedings.mlr.press/v235/cui24a.html>.
- [49] N. Tishby, F. C. Pereira und W. Bialek, *The Information Bottleneck Method*, 2000. arXiv: [physics/0004057](https://arxiv.org/abs/physics/0004057) [physics.data-an]. Adresse: <https://arxiv.org/abs/physics/0004057>.
- [50] A. Goyal, R. Islam, D. Strouse u. a., „InfoBot: Transfer and Exploration via the Information Bottleneck,” in *International Conference on Learning Representations (ICLR)*, 2019. Adresse: <https://arxiv.org/abs/1901.10902>.
- [51] Y. Pei und X. Hou, *Learning Representations in Reinforcement Learning: An Information Bottleneck Approach*, 2019. arXiv: [1911.05695](https://arxiv.org/abs/1911.05695) [cs.LG]. Adresse: <https://arxiv.org/abs/1911.05695>.
- [52] N. Tishby und N. Zaslavsky, „Deep Learning and the Information Bottleneck Principle,” *IEEE Information Theory Workshop*, 2015. Adresse: <https://arxiv.org/abs/1503.02406>.
- [53] Y. Jin, S. Wei, J. Yuan und X. Zhang, „Information-Bottleneck-Based Behavior Representation Learning for Multi-Agent Reinforcement Learning,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022. Adresse: <https://arxiv.org/abs/2109.14188>.
- [54] E. Wagstaff, F. B. Fuchs, M. Engelcke, M. A. Osborne und I. Posner, „Universal Approximation of Functions on Sets,” *Journal of Machine Learning Research*, Jg. 23, Nr. 151, S. 1–56, 2022. Adresse: <https://jmlr.org/papers/volume23/21-0730/21-0730.pdf>.
- [55] J. Frankle und M. Carbin, *The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks*, 2019. arXiv: [1803.03635](https://arxiv.org/abs/1803.03635) [cs.LG]. Adresse: <https://arxiv.org/abs/1803.03635>.
- [56] C. Li, H. Farkhoor, R. Liu und J. Yosinski, *Measuring the Intrinsic Dimension of Objective Landscapes*, 2018. arXiv: [1804.08838](https://arxiv.org/abs/1804.08838) [cs.LG]. Adresse: <https://arxiv.org/abs/1804.08838>.

- [57] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville und M. G. Bellemare, „Deep Reinforcement Learning at the Edge of the Statistical Precipice,“ in *Advances in Neural Information Processing Systems*, Bd. 34, Curran Associates, Inc., 2021, S. 29 304–29 320. arXiv: [2108.13264 \[cs.LG\]](#).
- [58] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup und D. Meger, „Deep Reinforcement Learning That Matters,“ in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI)*, Bd. 32, AAAI Press, 2018, S. 3207–3214. doi: [10.1609/aaai.v32i1.11694](#).